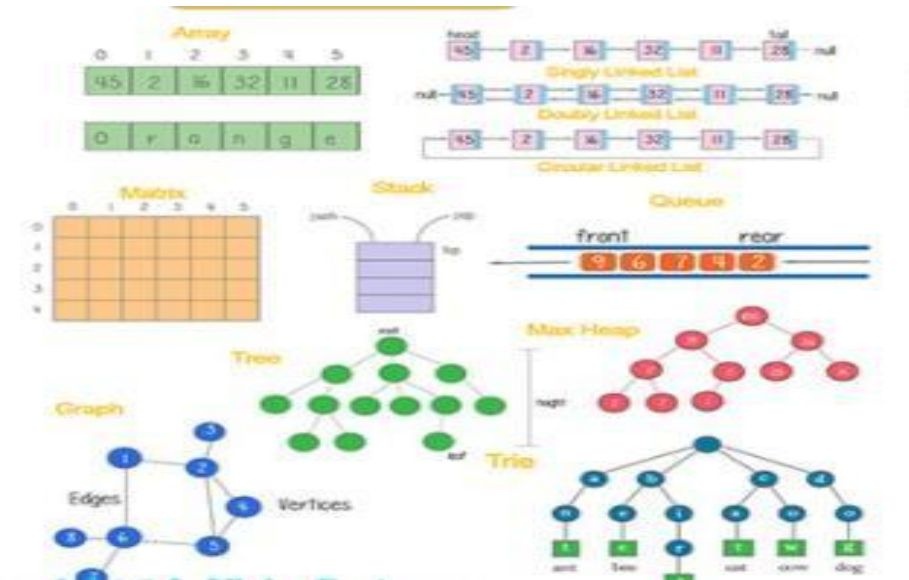


Data Structures

Dr. Elakkiya E

Asst. Professor,

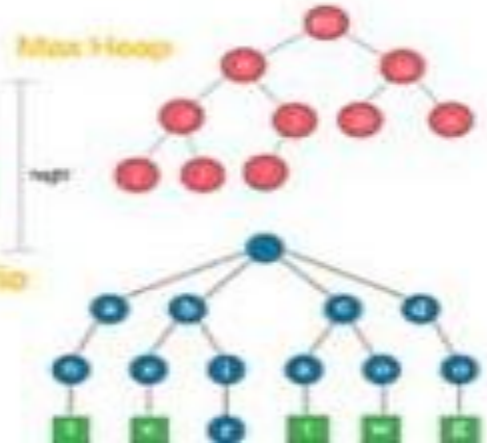
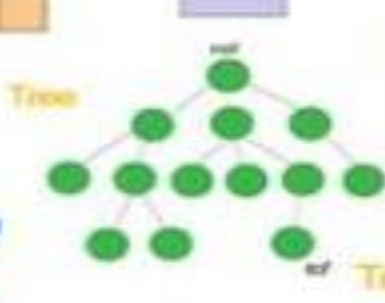
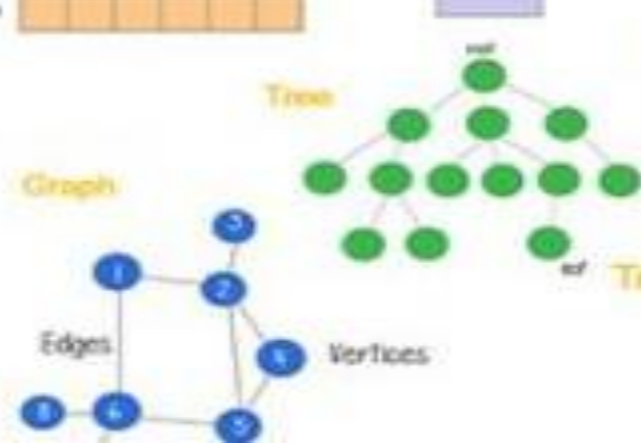
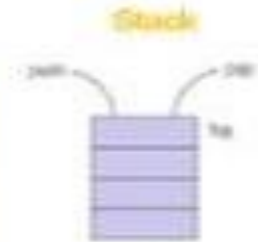
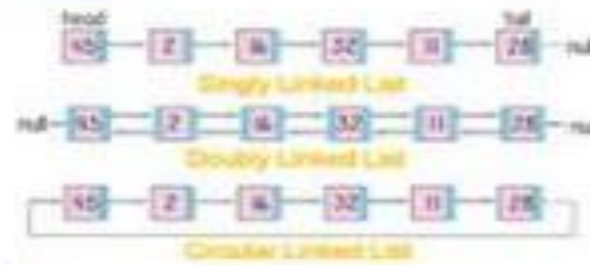
Dept. of CSE, SRM University AP



SRM
UNIVERSITY AP

Andhra Pradesh

Data structures



GRAPHS

UNIT-IV

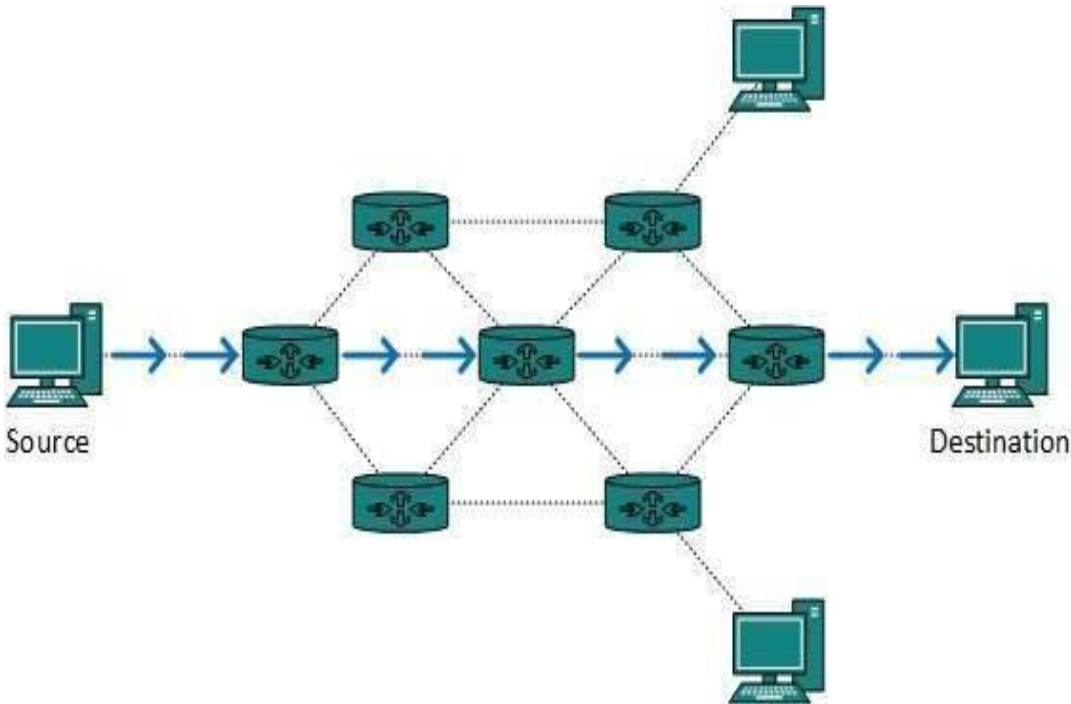
Graphs

Why Graphs ?

Graphs are widely used to model any situation where entities or things are related to each other in pairs.

- **Communication networks :**

- Routers in wired and wireless networks.



- **Social networks:**

- Graphs are also used in social network like linkedIn, facebook



- **Electrical circuits:**

- The networks may include paths in a telephone network or circuit network.

- **Task schedules:**

- We come across the Resource Allocation Graph where each process and resources are considered to be vertices in operating system

What is Graph ?

A **graph** G is defined as an ordered set (V, E) , where $V(G)$ represents the set of vertices and $E(G)$ represents the edges that connect these vertices.

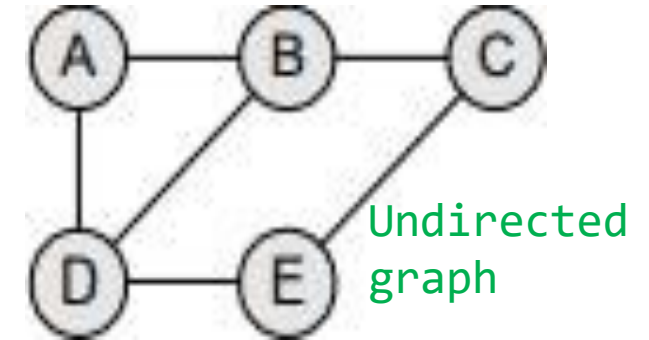
$V(G) = \{A, B, C, D, E\}$

$E(G) = \{(A, B), (B, C), (A, D), (B, D), (D, E), (C, E)\}$.

Note: There are **five vertices** or **nodes** and **six edges** in the graph.

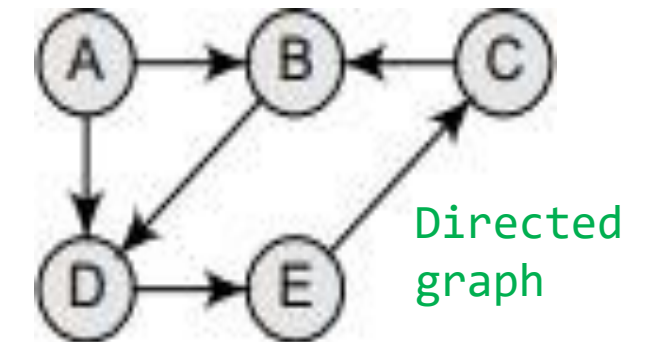
Undirected graph:

- ▶ **Edges do not have any direction** associated with them.
- ▶ If an edge is drawn between nodes A and B, then the nodes can be traversed from A to B as well as from B to A.



Directed graph (Digraph):

- ▶ **Edges form an ordered pair.**
- ▶ If there is an edge from A to B, then there is a path from A to B but not from B to A.
- ▶ The edge (A, B) is said to initiate from node A (also known as **initial node**) and terminate at node B (**terminal node**).

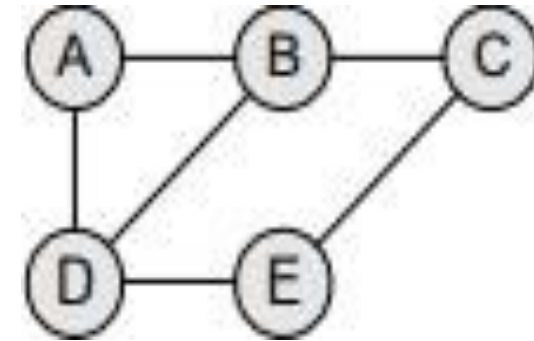


Graphs

Graph terminology

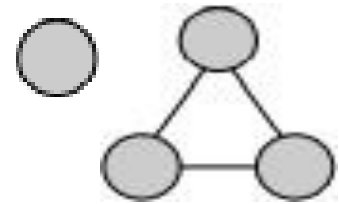
Adjacent nodes or neighbors:

For every edge, $e = (u, v)$ that connects nodes u and v , the nodes u and v are said to be the adjacent nodes or neighbors.



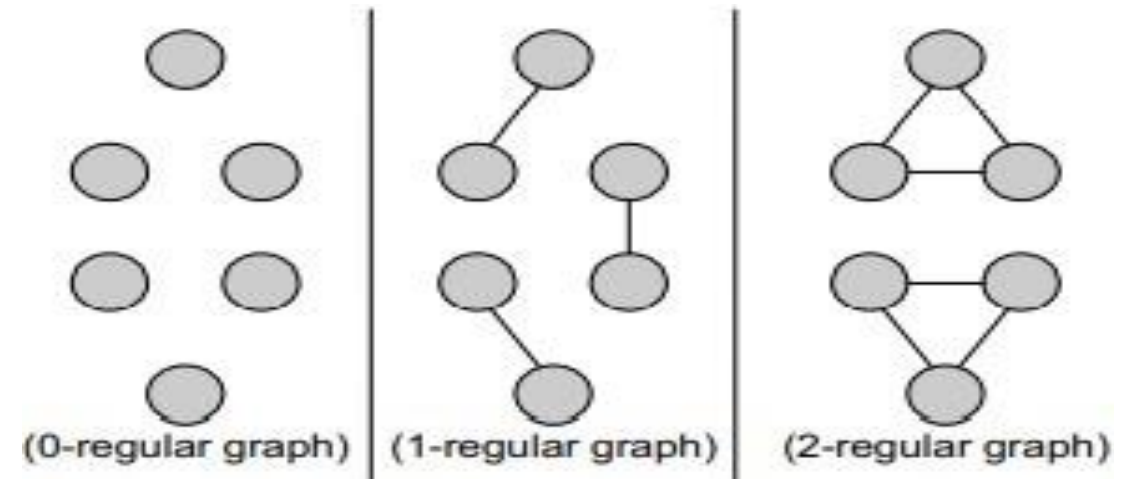
Degree of a node:

Degree of a node u , $\deg(u)$, is the total number of edges containing the node u . If $\deg(u) = 0$, it means that u does not belong to any edge and is known as an **isolated node**.



Regular graph:

- ▶ It is a graph where each vertex has the same number of neighbors.
- ▶ Every node has the same degree.
- ▶ A regular graph with vertices of degree k is called a k -regular graph or a regular graph of degree k .



Graph terminology

Path:

A path P written as $P = (v_0, v_1, v_2, \dots, v_n)$, of length n from a node u to v is defined as a sequence of $(n+1)$ nodes.

Here, $u = v_0$, $v = v_n$

v_{i-1} is adjacent to v_i for $i = 1, 2, 3, \dots, n$.

Ex: $P_1 = (U, W, X, Y, W, V)$ is a path

Closed path:

A path P is known as a closed path if the edge has the same end-points (if $v_0 = v_n$), Ex: $P_1 = (U, W, X, Y, W, V, U)$ is a closed path

Simple path:

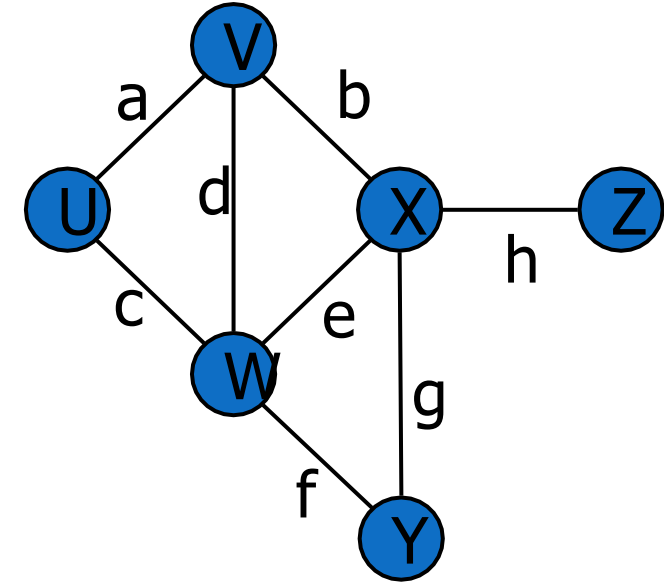
A path P is known as a simple path if all the nodes in the path are distinct with an exception that v_0 may be equal to v_n .

Ex: $P_1 = (V, X, Z)$ is a simple path, $P_2 = (U, W, X, Y, W, V)$ is not a simple path

If $v_0 = v_n$, then the path is called a closed simple path, Ex: $P_3 = (W, X, Y, W)$

► Cycle:

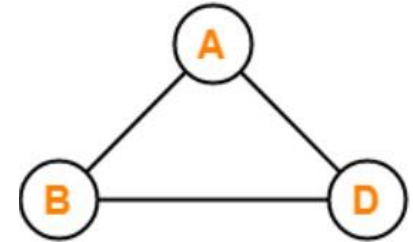
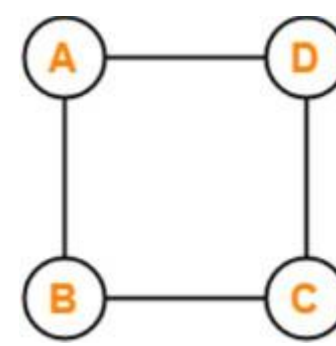
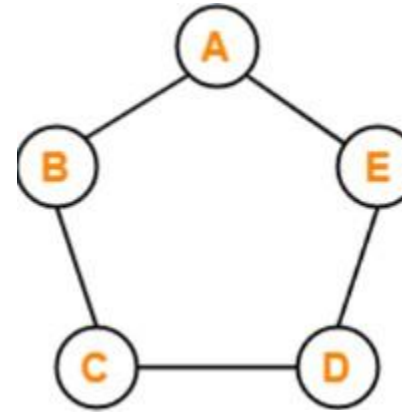
- A path in which the first and the last vertices are same.
- A simple cycle has no repeated edges or vertices (except the first and last vertices).



Graph terminology

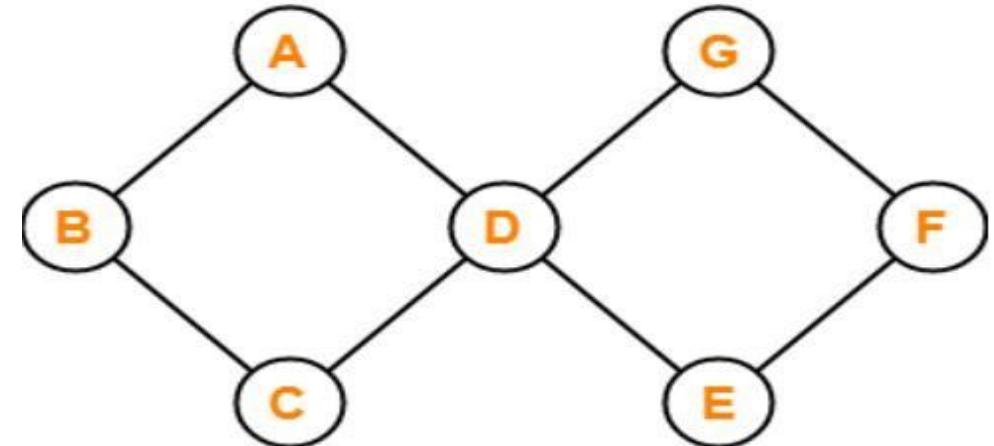
► Cycle Graph:

- If a graph with many vertices greater than three and edges form a cycle, then the graph is called a **cycle graph**.



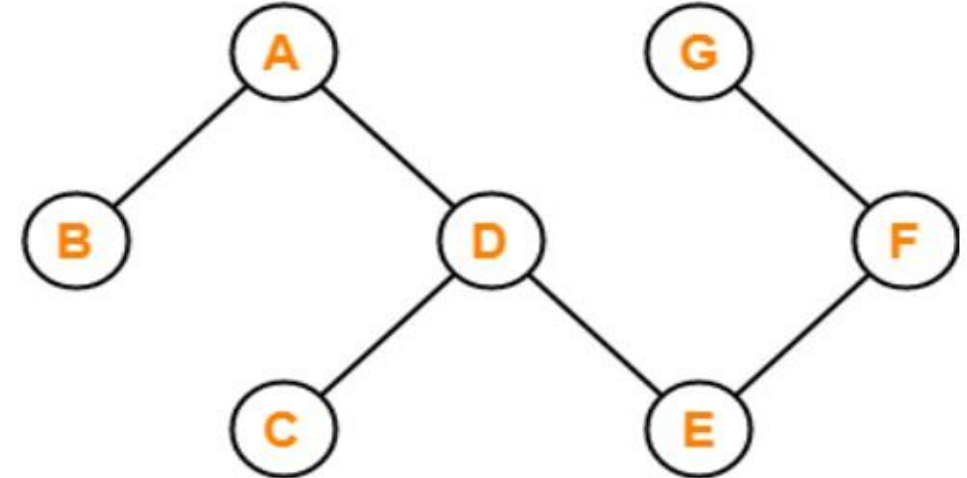
► Cyclic Graph:

- For a graph to be called a cyclic graph, it should consist of at least one cycle.
- If a graph has a minimum of one cycle present, it is called a cyclic graph.



► Acyclic Graph:

- A graph is called an acyclic graph if zero cycles are present, and an acyclic graph is the complete opposite of a cyclic graph.



Graph terminology

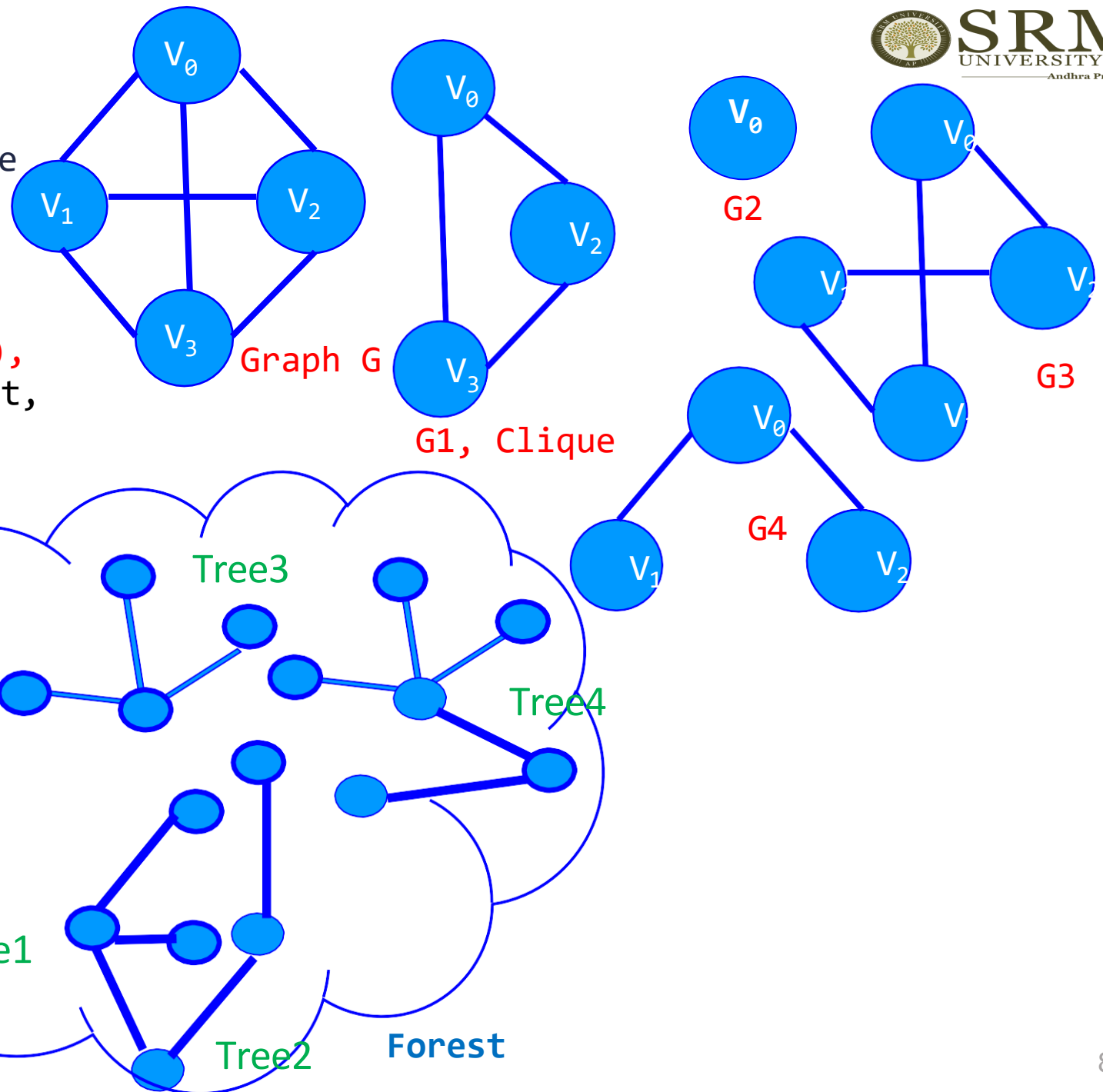
- ▶ **Subgraph:** Forming a graph with a subset of vertices and edges from the main graph.

- ▶ **Clique:**

- In an undirected graph $G = (V, E)$, clique is a subset of the vertex set, such that for every two vertices in the subset there is an edge that connects two vertices.

- ▶ **Trees and Forest**

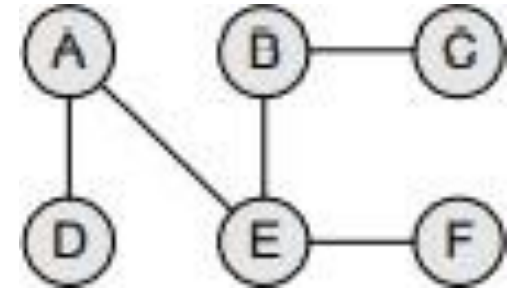
- **Tree** - connected graph without cycles
- **Forest** - collection of trees



Graph terminology

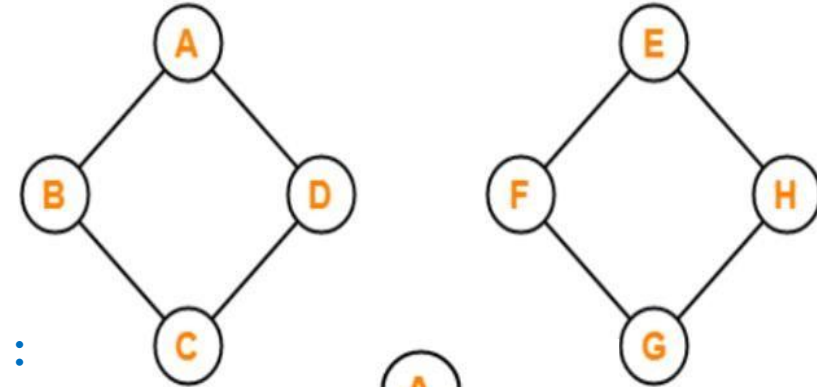
► Connected graph:

- For any two vertices (u, v) in V there is a path from u to v
- There are no isolated nodes in a connected graph.
- A connected graph that **does not have any cycle** is called a **tree**.



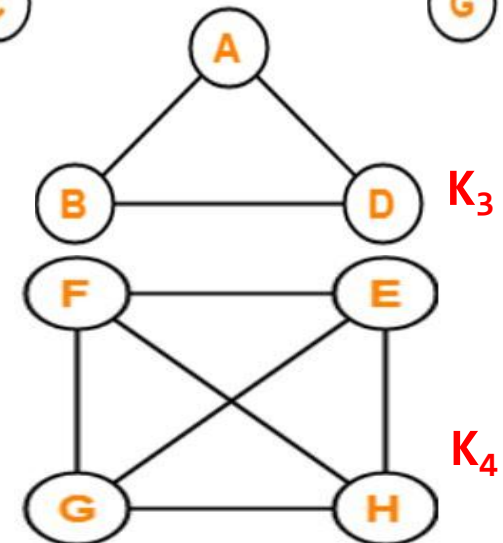
► Disconnected Graph

- A graph is said to be a disconnected graph where there does not exist any path between at least one pair of vertices.



► Complete or Fully connected or Strongly connected graph (K_n):

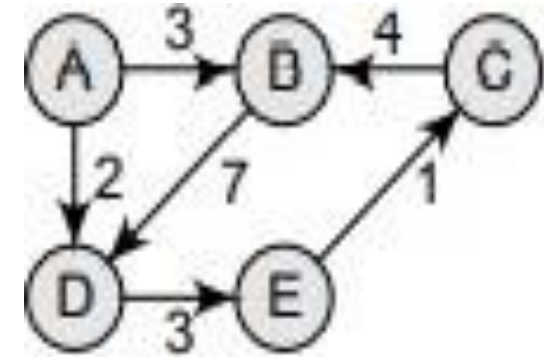
- G is said to be complete if all its nodes are fully connected.
- Fully connected: Each vertex directly connected every other vertex in the graph.
- A complete graph has $n(n-1)/2$ edges, where n is the number of nodes in G .



Graph terminology

► Labelled graph or weighted graph:

- A graph is said to be labelled if every edge in the graph is assigned some data.
- The edges of the graph are assigned some **weight or length**.



► Multiple edges or Parallel edges:

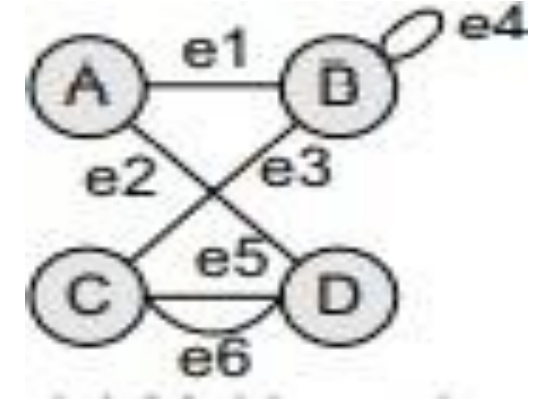
- Distinct edges which connect the same end-points are called multiple edges.
- $e = (u, v)$ and $e' = (u, v)$ are known as multiple edges of G .

► Loop:

- An edge that has identical end-points is called a loop ($e = (u, u)$).

► Multi-graph:

- A graph with multiple edges and/or loops is called a multi-graph.



► Size of a graph:

- The size of a graph is the total number of edges in it.

Terminology of a Directed Graph

► Out-degree of a node:

- The out-degree of a node u , written as $\text{outdeg}(u)$, is the number of edges that originate at u .

► In-degree of a node:

- The in-degree of a node u , written as $\text{indeg}(u)$, is the number of edges that terminate at u .

► Degree of a node:

- The degree of a node, written as $\text{deg}(u)$, is equal to the sum of in-degree and out-degree of that node.
- $\text{deg}(u) = \text{indeg}(u) + \text{outdeg}(u)$.

► Null Graph

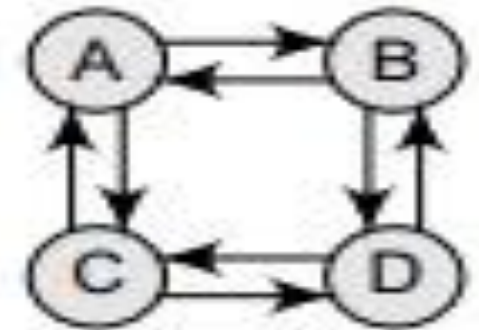
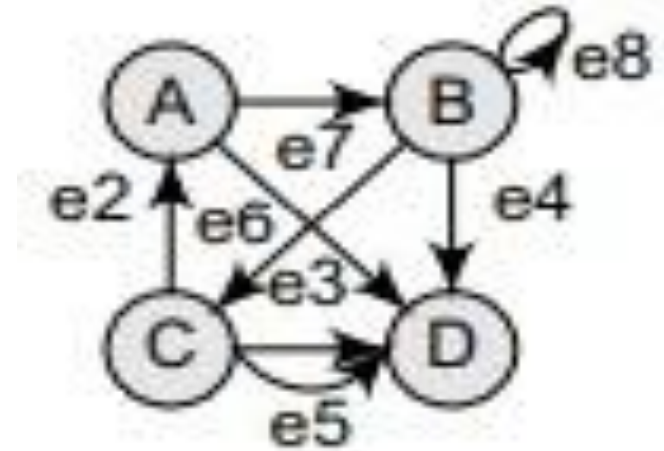
- The Null Graph is also known as the order zero graph. The term "*null graph*" refers to a graph with an empty edge set.

► Trivial Graph

- A graph is called a trivial graph if it has only one vertex present in it.

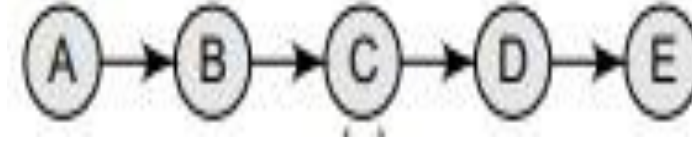


Trivial Graph



Null Graph

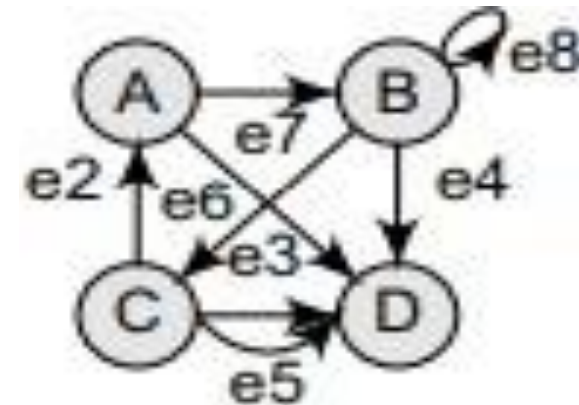
Terminology of a Directed Graph



- ▶ **Pendant vertex** (also known as leaf vertex): A vertex with degree one.
- ▶ **Cut vertex**: A vertex which when deleted would disconnect the remaining graph.

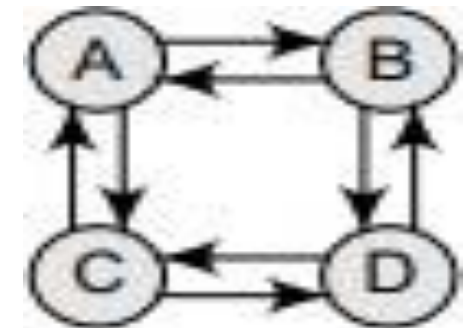
▶ **Reachability:**

- A node **v** is said to be reachable from node **u**, if and only if there exists a (directed) path from node **u** to node **v**.



▶ **Strongly connected directed graph:**

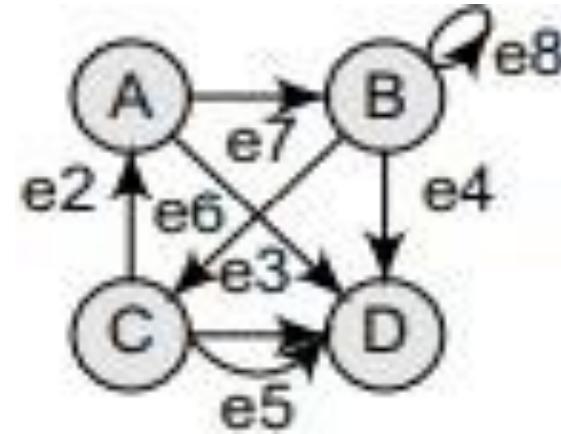
- A digraph is said to be strongly connected if and only if there exists a path between every pair of nodes in G.
- If there is a path from node **u** to **v**, then there must be a path from node **v** to **u**.



Terminology of a Directed Graph

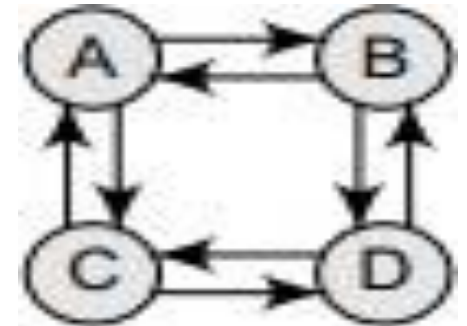
► Unilaterally connected graph:

- A digraph is said to be unilaterally connected if there exists a path between any pair of nodes u, v in G such that there is a path from u to v or a path from v to u , but not both.



► Simple directed graph:

- A directed graph G is said to be a simple directed graph if and only if it has no parallel edges.
- A simple directed graph may contain cycles with an exception that it cannot have more than one loop at a given node.



Representation of Graphs

- ▶ Two main ways of storing graphs in the computer's memory.
 - Sequential representation by using an adjacency matrix.
 - Linked representation by using an adjacency list that stores the neighbors of a node using a linked list.
- ▶ Adjacency Matrix Representation
 - An adjacency matrix is used to represent which nodes are adjacent to one another.
 - Two nodes are said to be adjacent if there is an edge connecting them.
 - In a directed graph G , if node v is adjacent to node u , then there is an edge from u to v .
 - For any graph G having n nodes, the adjacency matrix will have the dimension of $n \times n$.

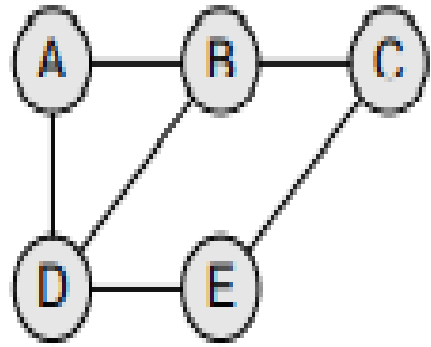
An entry a_{ij} in the adjacency matrix

1, if V_i is adjacent to V_j

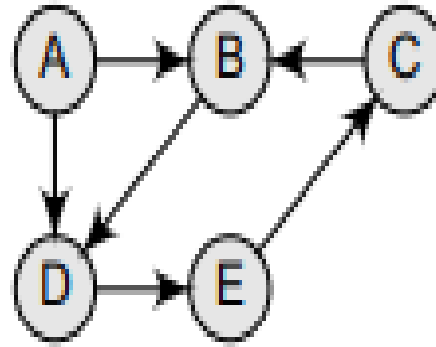
0, otherwise

Note: Since the matrix contains only 0s and 1s, it is called a bit matrix or a boolean matrix.

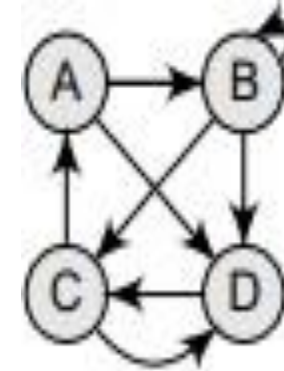
Adjacency Matrix Representation



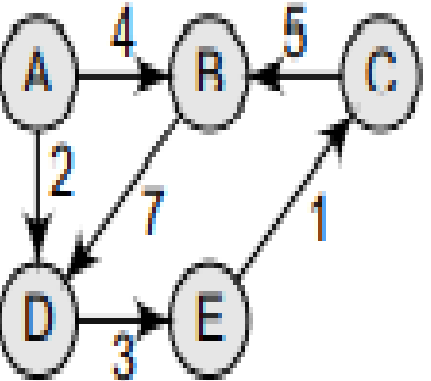
	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	1	0
C	0	1	0	0	1
D	1	1	0	0	1
E	0	0	1	1	0



	A	B	C	D	E
A	0	1	0	1	0
B	0	0	0	1	0
C	0	1	0	0	0
D	0	0	0	0	1
E	0	0	1	0	0



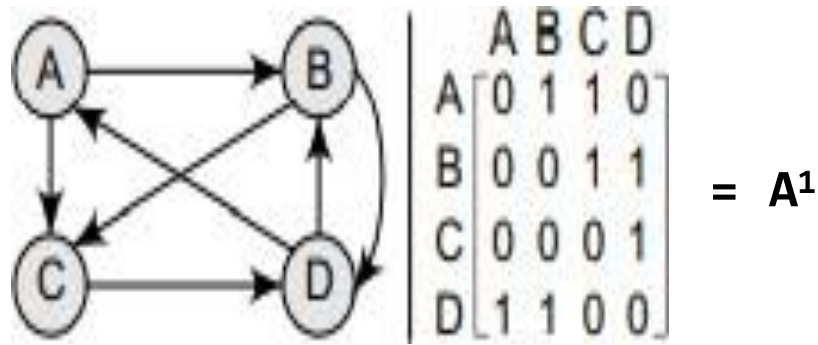
	A	B	C	D
A	0	1	0	1
B	0	1	1	1
C	1	0	0	1
D	0	0	1	0



	A	B	C	D	E
A	0	4	0	2	0
B	0	0	0	7	0
C	0	5	0	0	0
D	0	0	0	0	3
E	0	0	1	0	0

- ▶ For a **simple graph** (that has no loops), the adjacency matrix has **0s on the diagonal**.
- ▶ The adjacency matrix of an **undirected graph** is **symmetric**.
- ▶ The **memory use** of an adjacency matrix is **$O(n^2)$** , where n is the number of nodes in the graph.
- ▶ **Number of 1s** (or non-zero entries) in an adjacency matrix is equal to the **number of edges** in the graph.
- ▶ The adjacency matrix for a **weighted graph** contains the **weights** of the edges connecting the nodes.

Path Matrix



$$A^2 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix}$$

- From adjacency matrix A^1 , an entry 1 in the i th row and j th column means that there exists a path of length 1 from v_i to v_j .

For A^2 we perform $(a_{ij})^2 = \sum a_{ik} * a_{kj}$

Any entry $a_{ij} = 1$ if $a_{ik} = a_{kj} = 1$.

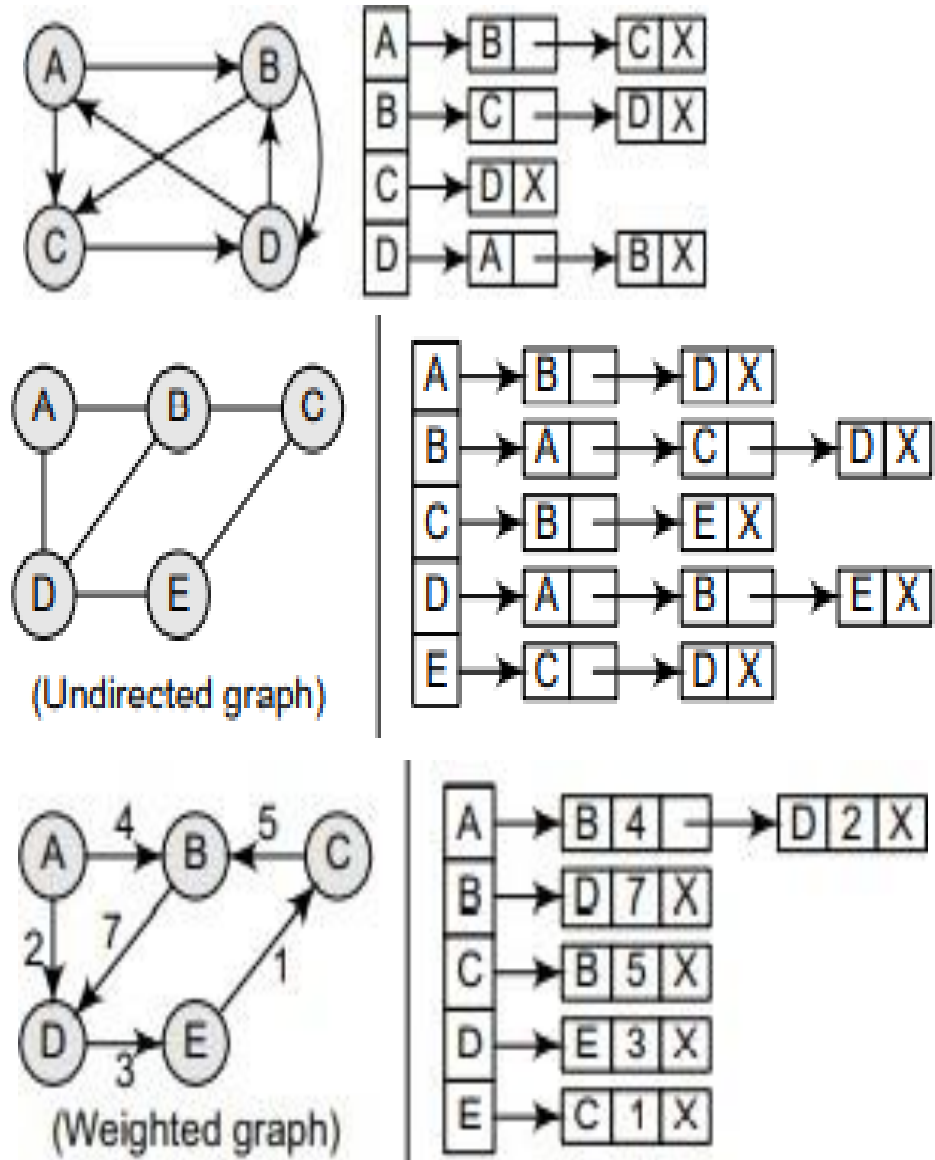
- If there is an edge (v_i, v_k) and (v_k, v_j) , then there is a path from v_i to v_j of length 2.
- In general, every entry in the i th row and j th column of A^n gives the number of paths of length n from node v_i to v_j .
- The path matrix is used to show whether there exists a simple path from node v_i to v_j or not.

Path matrix can be given as,

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix} + \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 2 & 3 \\ 2 & 3 & 3 & 3 \\ 1 & 2 & 2 & 2 \\ 3 & 4 & 4 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Path matrix

Adjacency List Representation



- ▶ Every node is linked to its own list that contains the names of all other nodes that are adjacent to it.
- ▶ **Key advantages**
 - It is often used for storing graphs that have a small-to-moderate number of edges.
 - Adjacency list is preferred for **sparse graphs**; otherwise, adjacency matrix is a good choice.
 - Adding new nodes in **G** is easy and straightforward when **G** is represented using an adjacency list.
 - Adding new nodes in an adjacency matrix is a difficult task, as the size of the matrix needs to be changed and existing nodes may have to be reordered.
- ▶ For a directed graph, the sum of the lengths of all adjacency lists is equal to the number of edges.
- ▶ For an undirected graph, the sum of the lengths of all adjacency lists is equal to twice the number of edges.

- ▶ Graph traversal is a method of examining the nodes and edges of the graph.
- ▶ Traversal of graph $G=(V,E)$ and vertex v is finding all w in V , such that w connects v .
 - **Breadth First Search (BFS)** (level order tree traversal in trees)
 - **Depth First Search (DFS)** (preorder tree traversal in trees)
- ▶ Breadth-first search algorithm uses a queue as an auxiliary data structure to store nodes for processing.
- ▶ Depth-first search algorithm uses a stack.

Breadth First Search (BFS)

- ▶ BFS begins at the root node and explores all the neighboring nodes.
 - Then for each of the nearest nodes, the algorithm explores their unexplored neighbor nodes, and so on, until it finds the goal.

Algorithm

Step 1: Set **STATUS=1** (ready state)

for each node in **G**

Step 2: **Enqueue** the starting node **A** and set its **STATUS=2** (waiting state)

Step 3: Repeat Steps 4 and 5 until **QUEUE** is empty

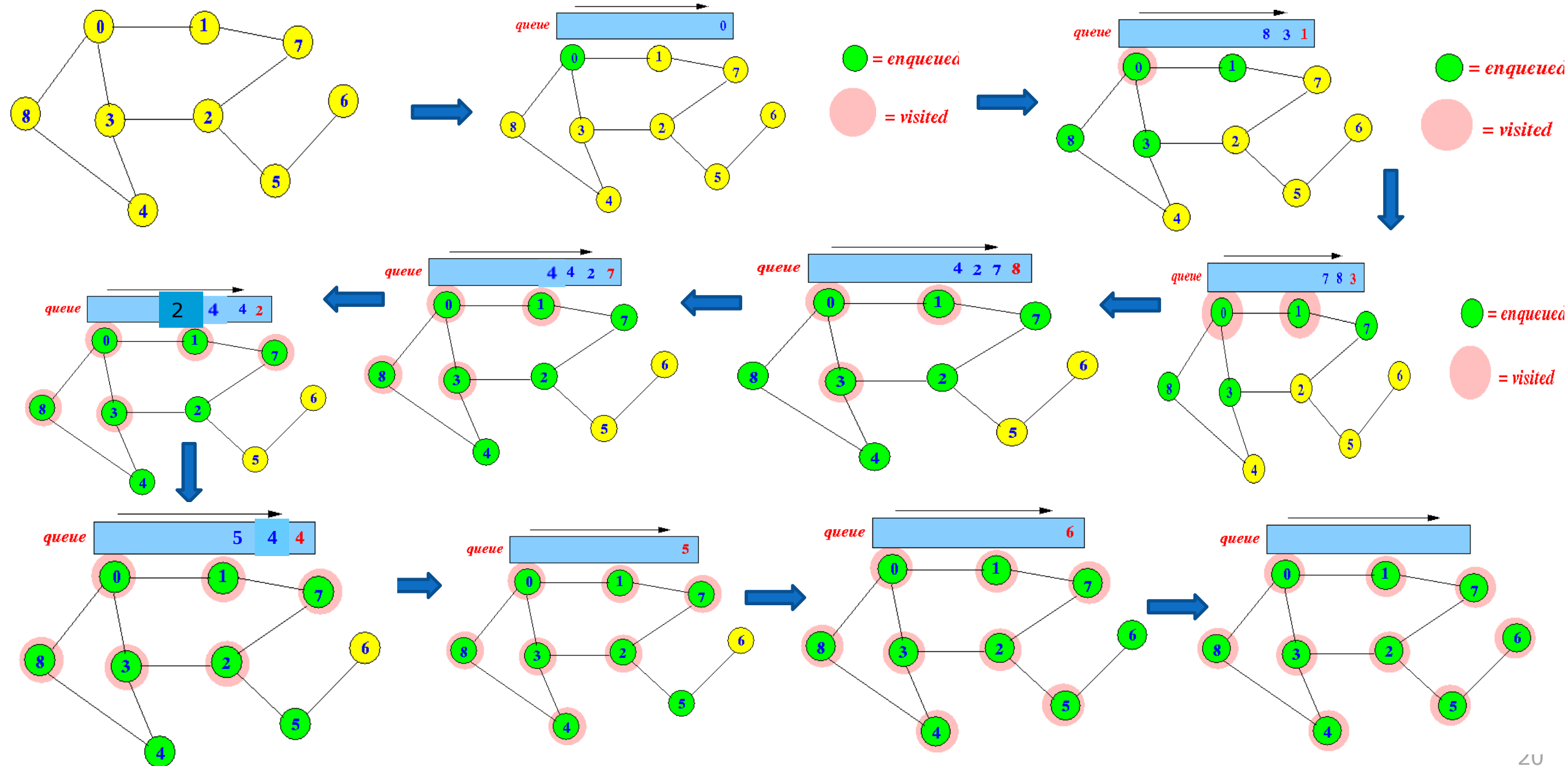
Step 4: **Dequeue** a node **N**. Process it and set its **STATUS=3** (processed state).

Step 5: **Enqueue** all the neighbors of **N** that are in the ready state (whose STATUS=1) and set their STATUS=2 (waiting state)

[End of Loop]

Step 6: EXIT

Breadth First Search (BFS)



Breadth First Search (BFS)

```
void breadth_first_search(int adj[][MAX],int visited[],int start)
{
    int queue[MAX],rear = -1,front =- 1, i;
    queue[++rear] = start;
    visited[start] = 1;
    while(rear != front)
    {
        start = queue[++front];
        printf("%c \t",start + 65);
        for(i = 0; i < MAX; i++)
        {
            if(adj[start][i] == 1 && visited[i] == 0)
            {
                queue[++rear] = i;
                visited[i] = 1;
            }
        }
    }
}
```

```
#define MAX 10
int main()
{
    int visited[MAX] = {0};
    int adj[MAX][MAX], i, j;
    printf("Enter adjacency matrix:");
    for(i = 0; i < MAX; i++)
        for(j = 0; j < MAX; j++)
            scanf("%d", &adj[i][j]);
    breadth_first_search(adj,visited,0);
    return 0;
}
```

- The Time complexity of BFS is $O(|V| + |E|)$ when Adjacency List is used and $O(|V|^2)$ when Adjacency Matrix is used.

Depth First Search (DFS)

- ▶ DFS progresses by expanding the starting node of G
 - then going deeper and deeper until the goal node is found, or
 - until a node that has no children is encountered

Algorithm

Step 1: Set **STATUS=1** (ready state)

for each node in **G**

Step 2: **Push** the starting node **A** on the stack and set its **STATUS=2** (waiting state)

Step 3: Repeat Steps 4 and 5 until **STACK** is empty

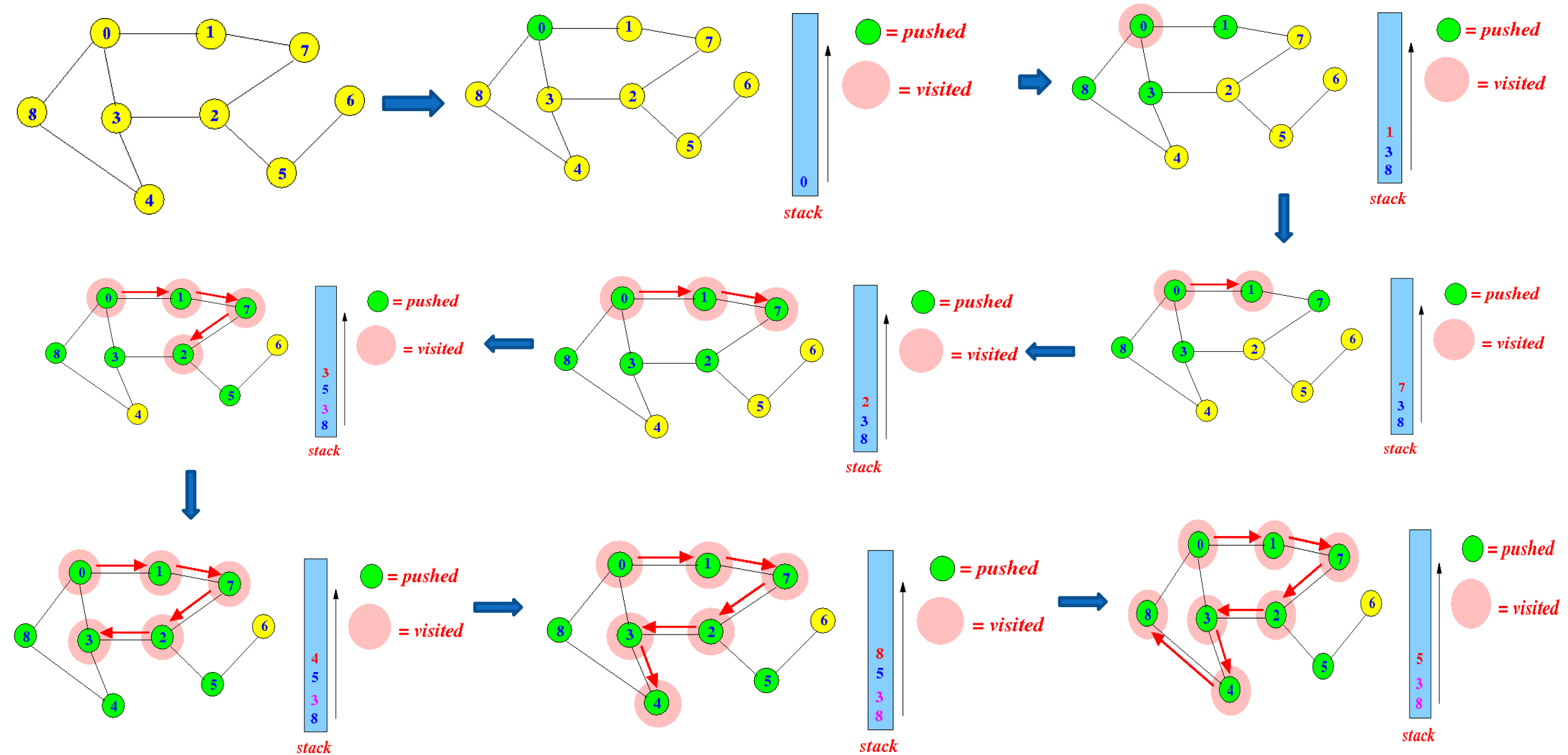
Step 4: **Pop** the **top** node **N**. Process it and set its **STATUS=3** (processed state)

Step 5: **Push** on the stack all the neighbors of **N** that are in the **ready state** (whose **STATUS=1**) and set their **STATUS=2** (waiting state)

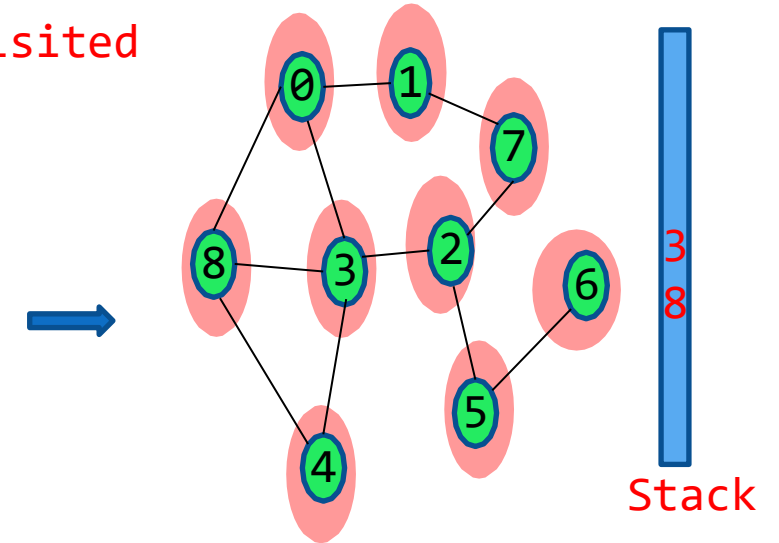
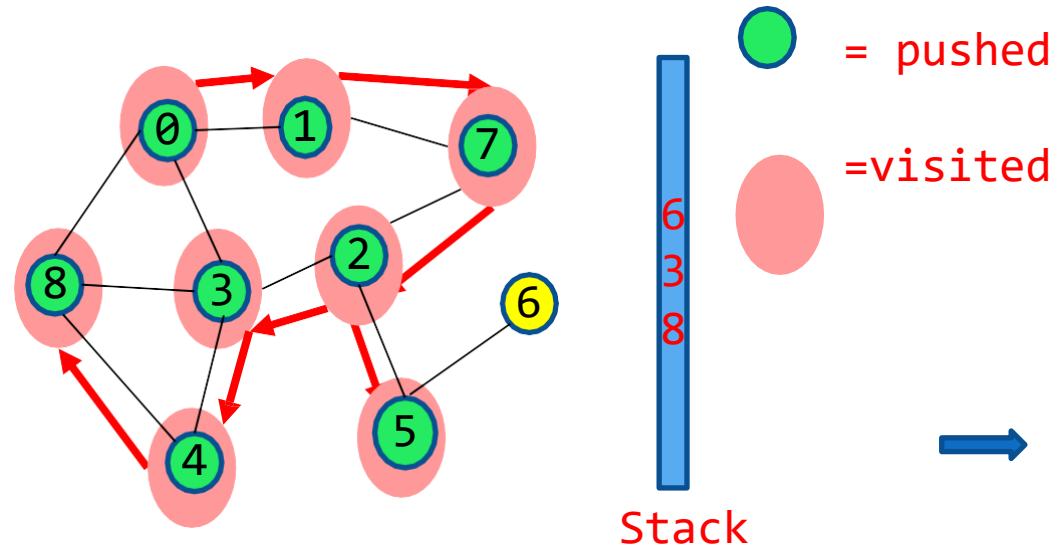
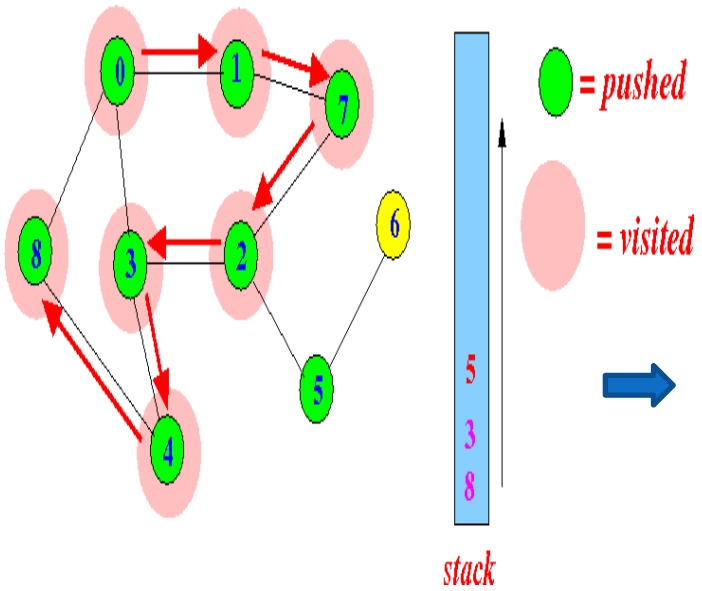
[End of Loop]

Step 6: EXIT

Depth First Search (DFS)



Depth First Search (DFS)



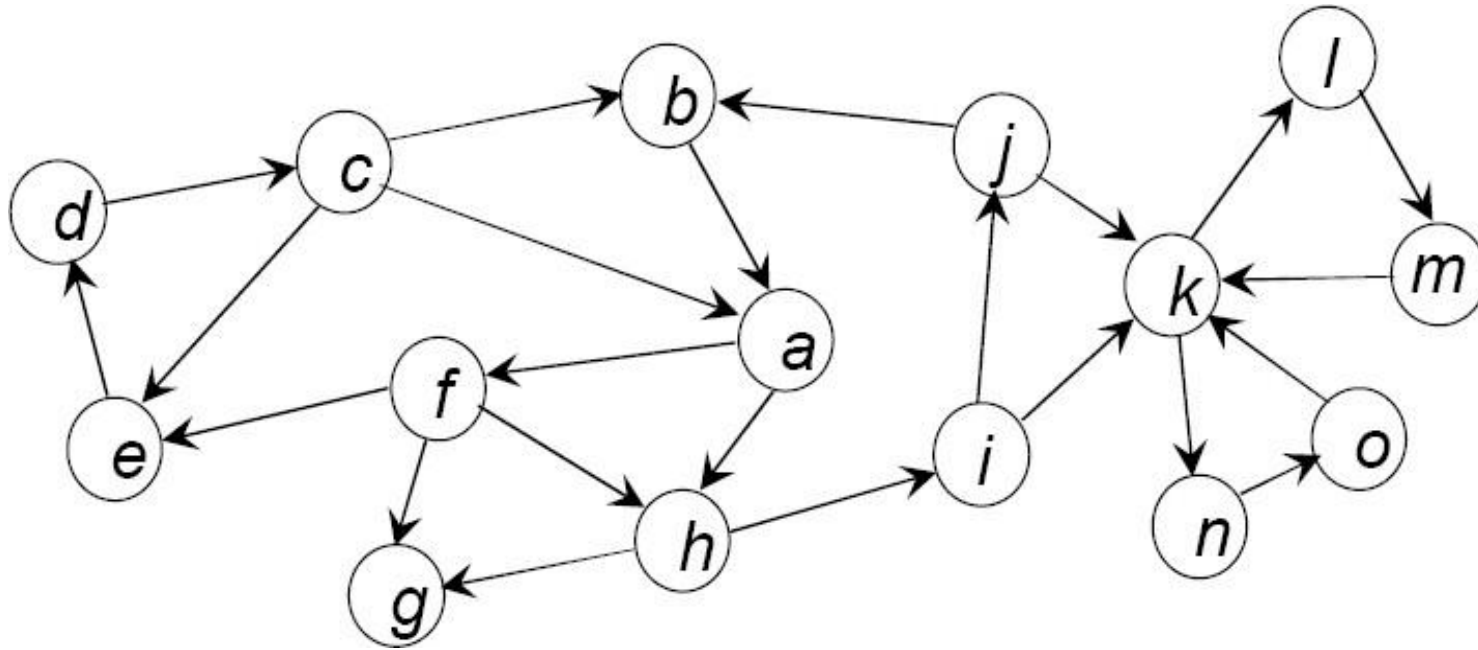
DFS = 0, 1, 7, 2, 3, 4, 8, 5, 6

Depth First Search (DFS)

```
void depth_first_search(int adj[][MAX],int visited[],int start)
{
    int stack[MAX], top = -1, i;
    printf("%c-",start + 65);
    visited[start] = 1;
    stack[++top] = start;
    while(top != -1){
        start = stack[top];
        for(i = 0; i < MAX; i++){
            if(adj[start][i] && visited[i] == 0)
            {
                stack[++top] = i;
                printf("%c-", i + 65);
                visited[i] = 1;
                break;
            }
        }
        if(i == MAX)
            top--;
    }
}
```

```
#define MAX 5
int main()
{
    int adj[MAX][MAX];
    int visited[MAX] = {0}, i, j;
    printf("\nEnter adjacency matrix: ");
    for(i = 0; i < MAX; i++)
        for(j = 0; j < MAX; j++)
            scanf("%d", &adj[i][j]);
    printf("DFS Traversal: ");
    depth_first_search(adj,visited,0);
    printf("\n");
    return 0;
}
```

- The Time complexity of DFS is $O(|V| + |E|)$ when Adjacency List is used and $O(|V|^2)$ when Adjacency Matrix is used.



Find Breadth-First and Depth-First Traversal starting from node 'a'

Breadth-first search: a f h e g i d j k c l n b m o

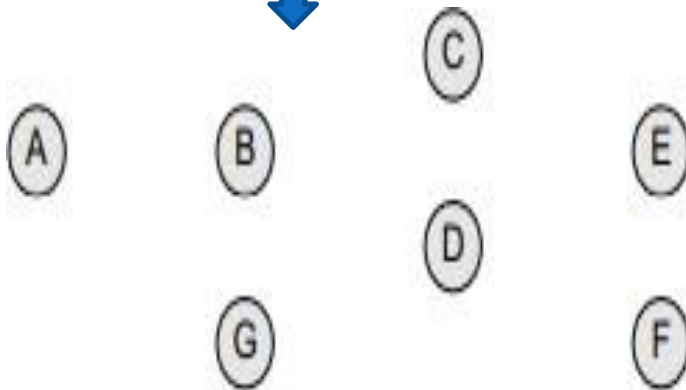
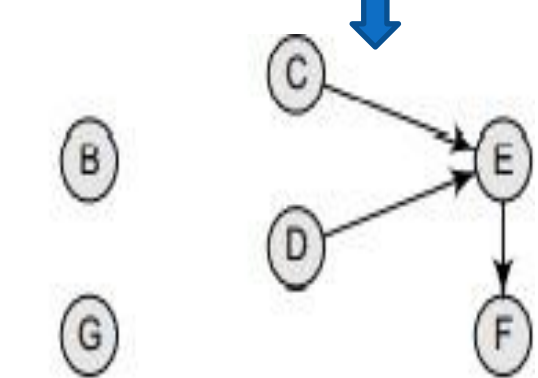
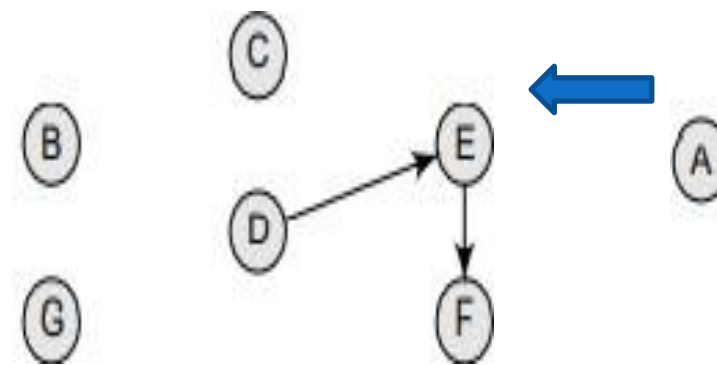
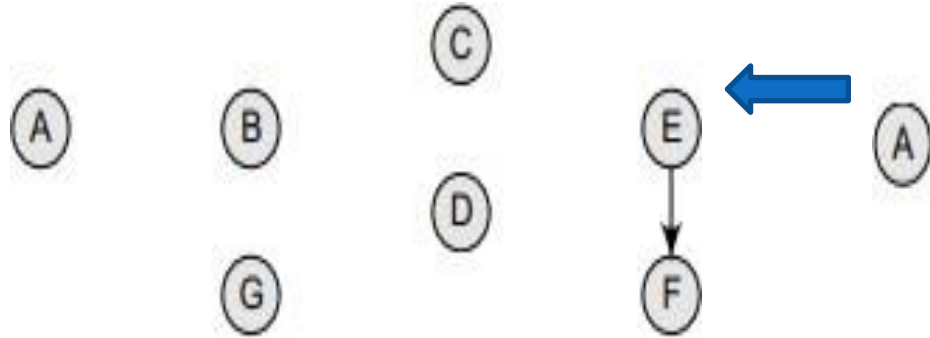
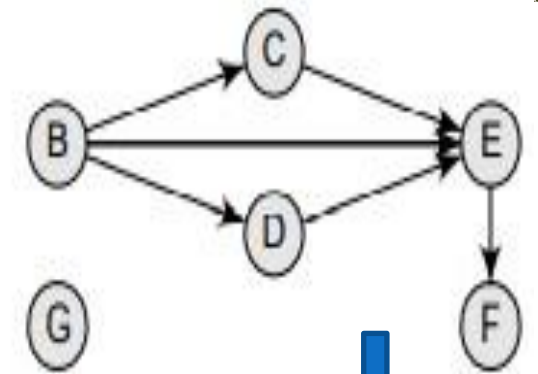
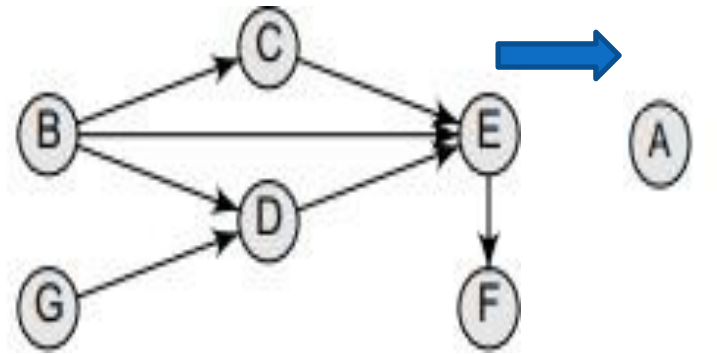
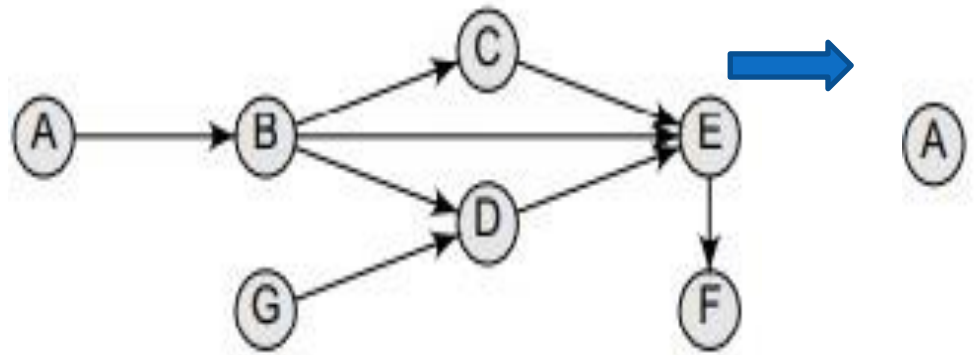
Depth-first search: a f e d c b g h i j k l m n o

- **Directed Acyclic Graph (DAG)** is a directed graph that does not contain the cycle
- A **topological sort** of a DAG G is an ordering of the vertices of G such that if G contains an edge (u, v) , then **u appears before v in the ordering.**
- Note that topological sort is possible only on DAGs
- **Every DAG has one or more number of topological sorts.**
- Topological sorting is widely used in scheduling applications, jobs, or tasks.

Steps in Topological sort Algorithm

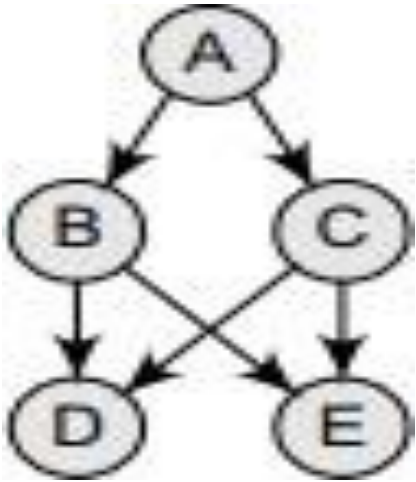
- Step-1: Selecting a node N with zero in-degree
- Step-2: Deleting (Process or display) N from the graph along with its edges
- Step-3: Repeat steps 1 and 2

Topological sorting



Topological sort order: A, G, B, C, D, E, F

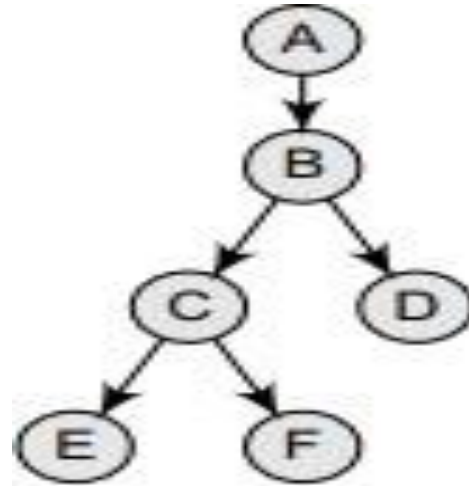
Topological sorting



Topological sort orders:

A, B, C, D, E

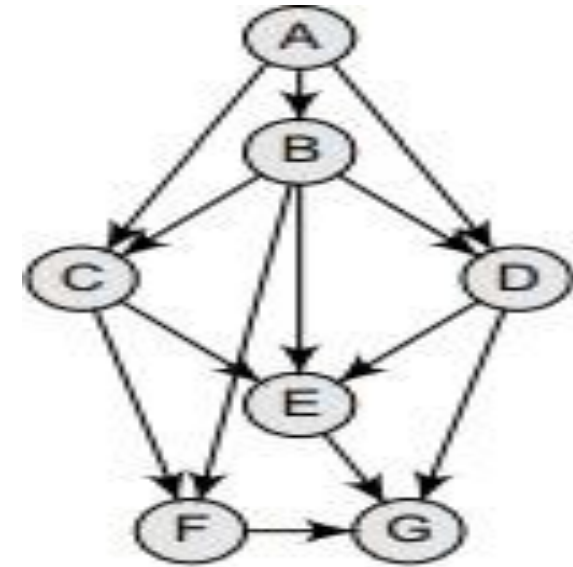
A, B, C, E, D



Topological sort orders:

A, B, D, C, E, F

A, B, D, C, F, E



Topological sort orders:

A, B, C, F, D, E, G

A, B, C, D, E, F, G

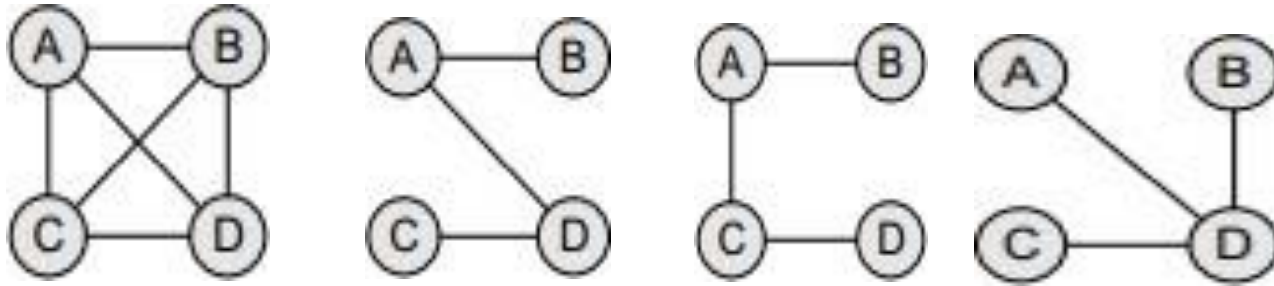
Shortest path algorithms

Different algorithms to calculate the shortest path between the vertices of a graph G.

- ▶ Minimum spanning tree
 - ▶ Dijkstra's algorithm
 - ▶ Floyd-Warshall's algorithm
- Uses Adjacency list
- Uses Adjacency matrix

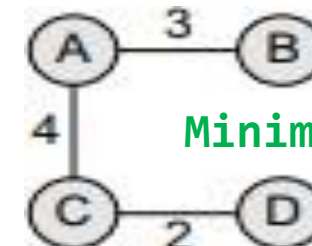
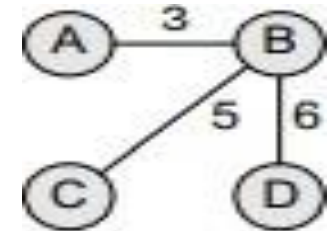
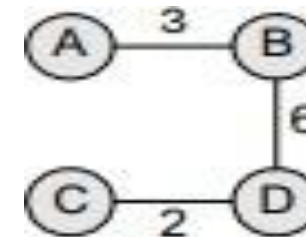
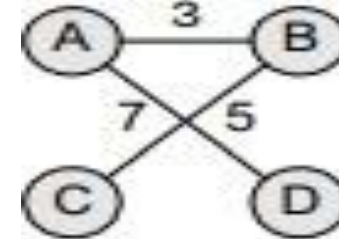
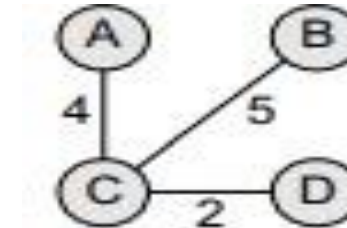
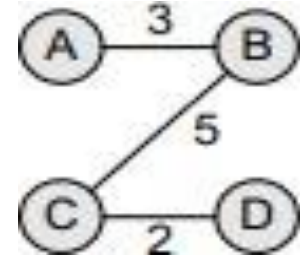
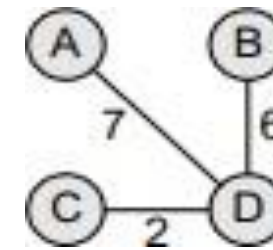
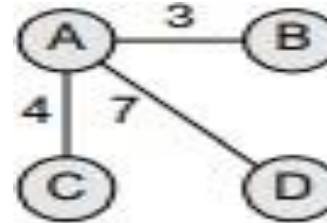
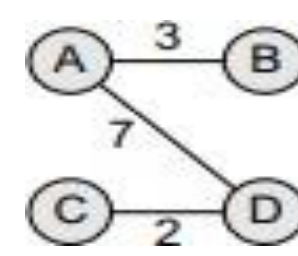
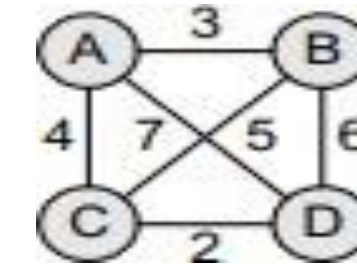
Minimum Spanning trees

- A spanning tree of a connected, undirected graph G is a sub-graph of G which is a tree that connects all the vertices together
- A graph G can have many different spanning trees.



Weight of a spanning tree is the sum of the weights of the edges in that spanning

A minimum spanning tree (MST) is defined as a spanning tree with weight less than or equal to the weight of every other spanning tree



Minimum Spanning Tree (Weight = 9)

Tree vertices: Vertices that are a part of the minimum spanning tree T.

Fringe vertices: Vertices that are currently not a part of T, but are adjacent to some tree vertex.

Unseen vertices: Vertices that are neither tree vertices nor fringe vertices fall under this category

Algorithm

Step 1: Select a starting vertex

Step 2: Repeat Steps 3 and 4 until there are fringe vertices

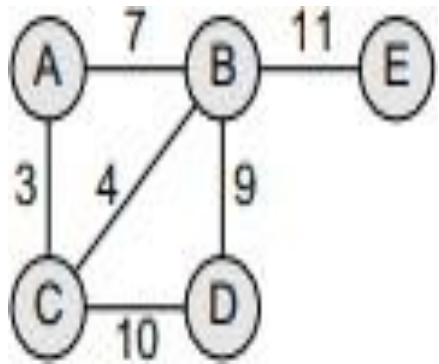
Step 3: Select an edge e connecting the tree vertex and fringe vertex that has minimum weight

Step 4: Add the selected edge and the vertex to the minimum spanning tree T

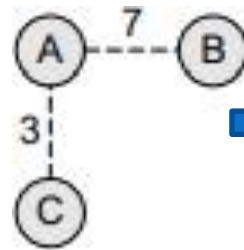
[END OF LOOP]

Step 5: EXIT

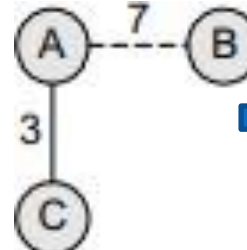
Prim's Algorithm



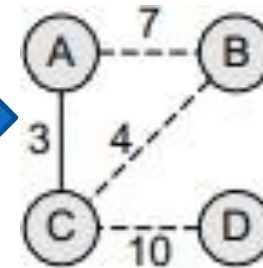
Step 1



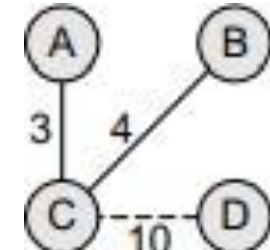
Step 2



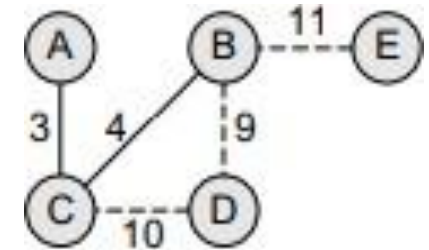
Step 3



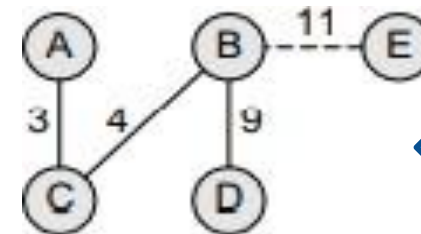
Step 4



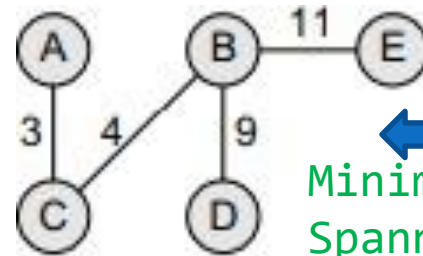
Step 5



Step 6

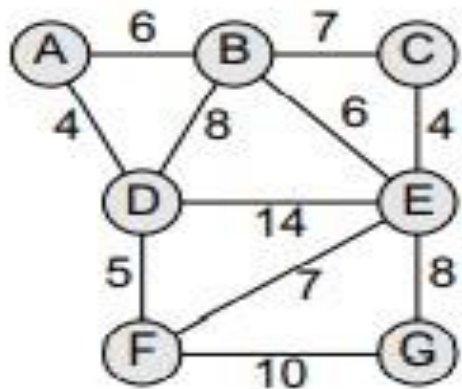


Step 7



Step 8

Minimum
Spanning
Tree

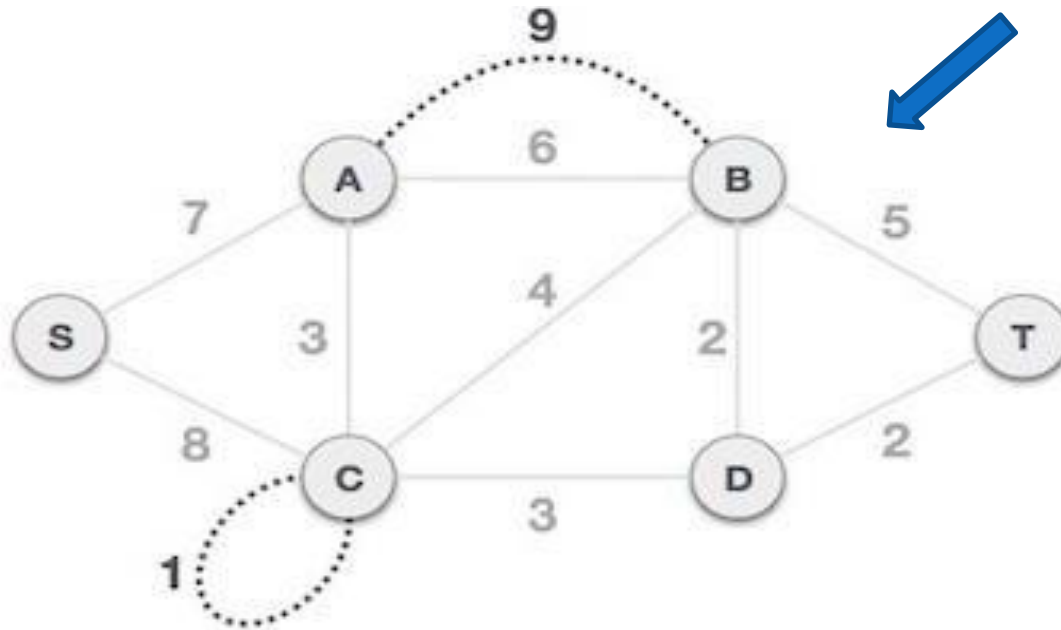
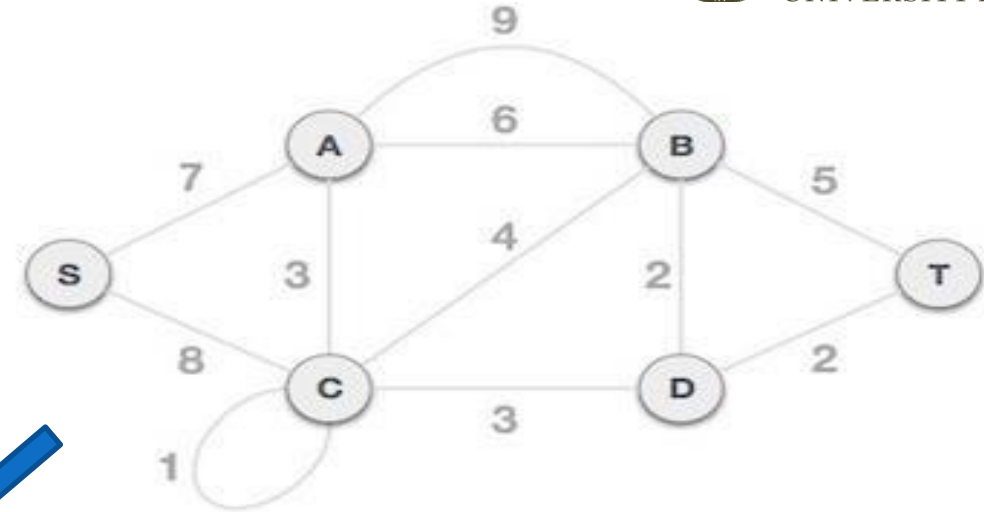


Find Minimum Spanning Tree:

Kruskal's Algorithm

Algorithm treats the graph as a forest and every node it has as an individual tree.

A tree connects to another only and only if, it has the least cost among all available options and does not violate MST properties.



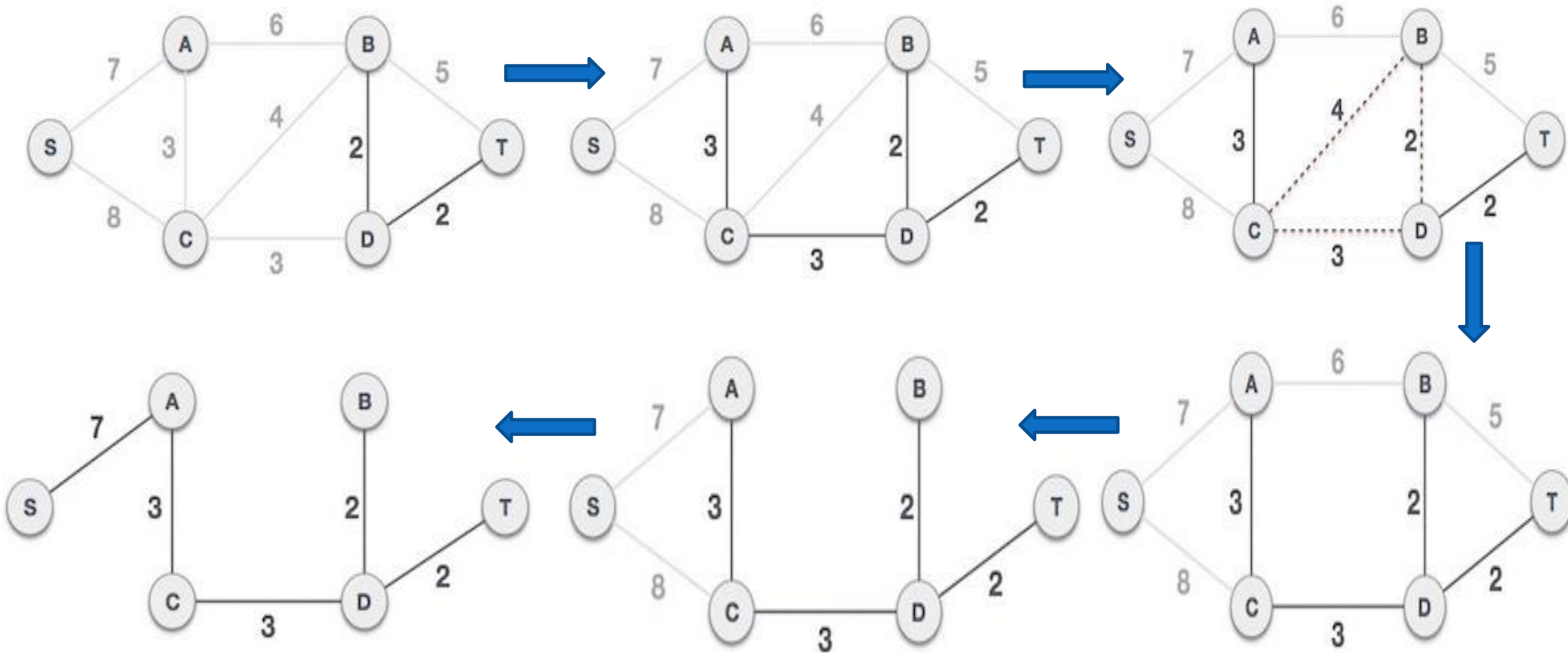
Step 1 - Remove all loops and Parallel Edges

Step 2 - Arrange all edges in their increasing order of weight

B, D	D, T	A, C	C, D	C, B	B, T	A, B	S, A	S, C
2	2	3	3	4	5	6	7	8

Kruskal's Algorithm

Step 3 - Add the edge which has the least weightage



Dijkstra's Algorithm (Single Source Shortest Path Algorithm)

Step-1: Maintain a list of unvisited vertices.

Step-2: Assign a vertex as “source” and also allocate a maximum possible cost (infinity) to every other vertex. (The cost of the source to itself will be zero)

Step-3: In every step, try to minimize the cost for each vertex.

Step-4: For every unvisited neighbor (V2, V3) of the current vertex (V1) calculate the new cost from V1.

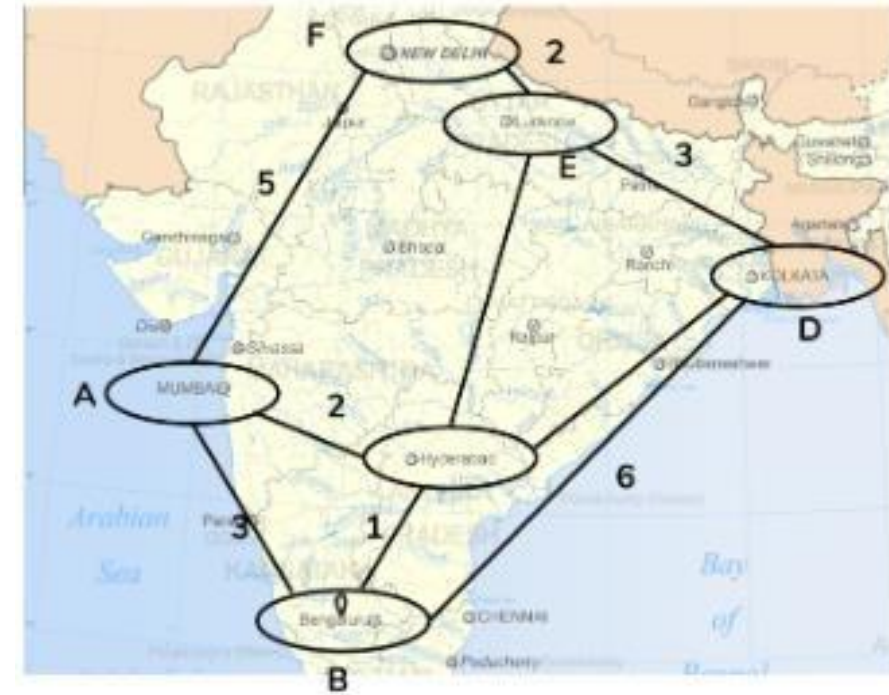
Step-5: The new cost of V2 is calculated as :

$\text{Minimum(existing cost of V2 , (sum of cost of V1 + the cost of edge from V1 to V2))}$

Step-6: Mark the current node V1 as visited and remove it from the unvisited list.

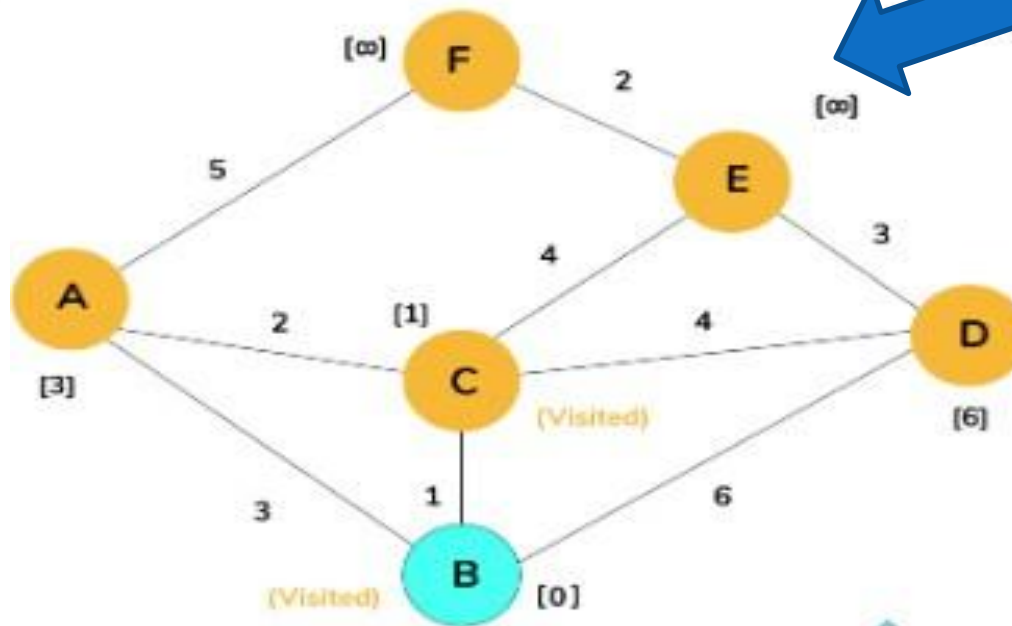
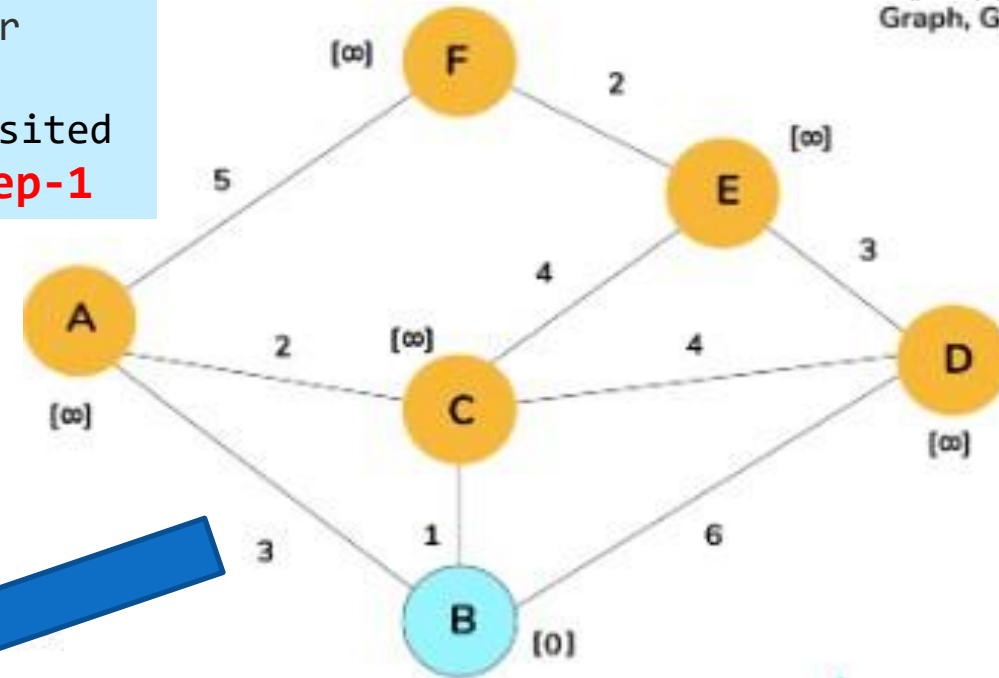
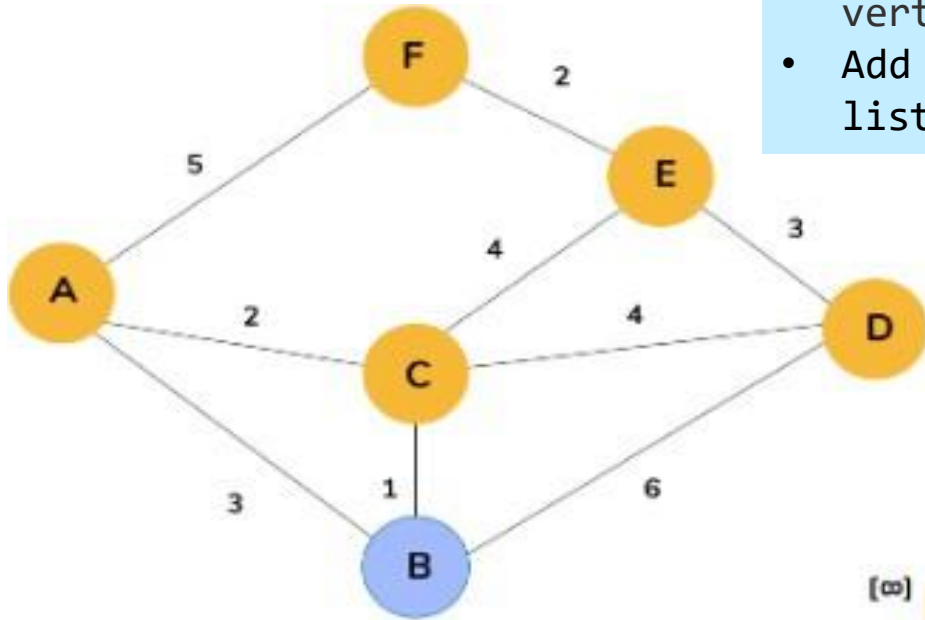
Step-7: Select next vertex with smallest cost from the unvisited list and repeat from step 4.

Step-8: The algorithm finally ends when there are no unvisited nodes left.



Dijkstra's Algorithm

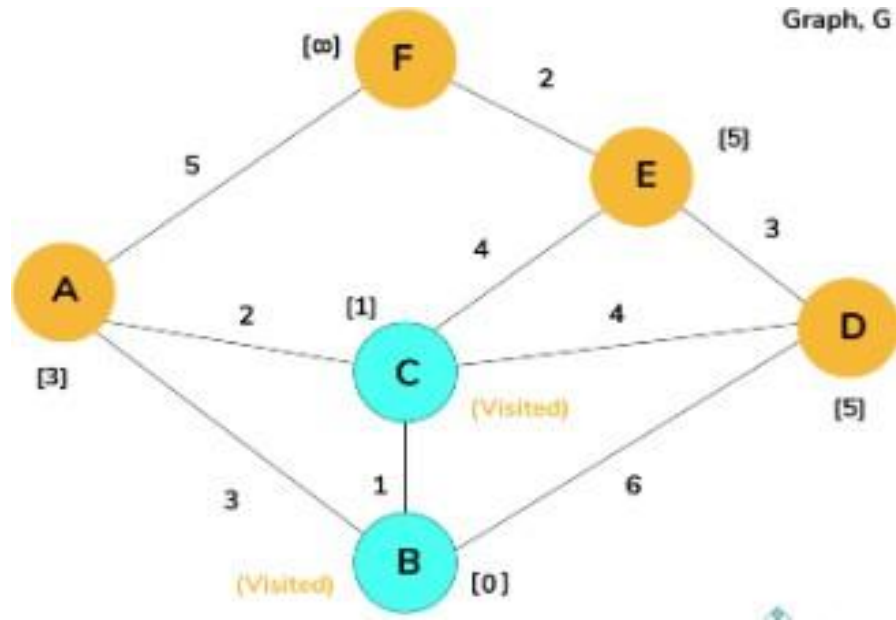
- Assign cost of 0 to source vertex and ∞ (Infinity) to all other vertices
 - Add all the vertices to unvisited list
- Step-1**



- Step-2**
- For neighbor A:
cost = Minimum(∞ , $0+3$) = 3
 - For neighbor C:
cost = Minimum(∞ , $0+1$) = 1
 - For neighbor D:
cost = Minimum(∞ , $0+6$) = 6

Dijkstra's Algorithm

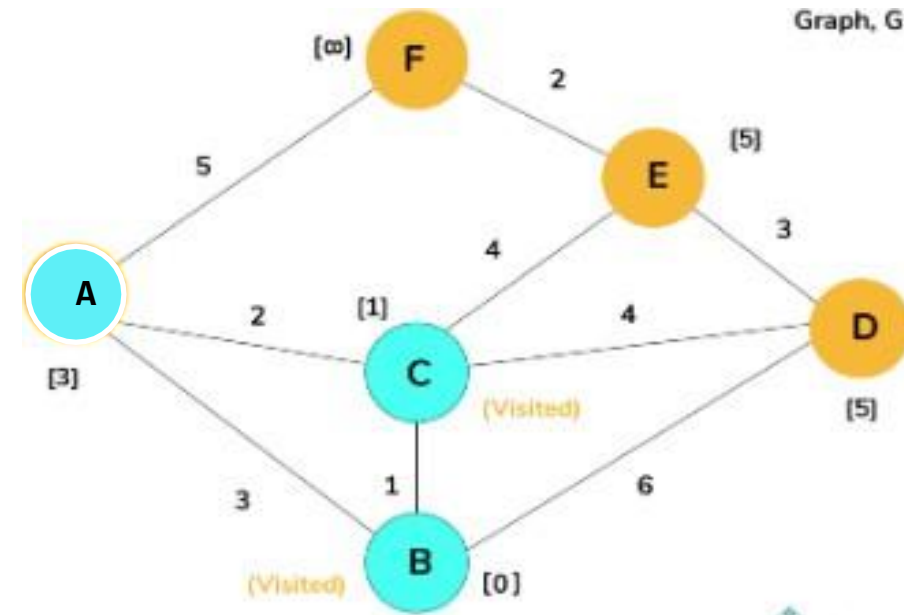
Step-3: Select next vertex with smallest cost from the unvisited list. (C)



- For neighbor A: cost = Minimum(3 , 1+2) = 3
- For neighbor D: cost = Minimum(6 , 1+4) = 5
- For neighbor E: cost = Minimum(∞ , 1+4) = 5

Step-4

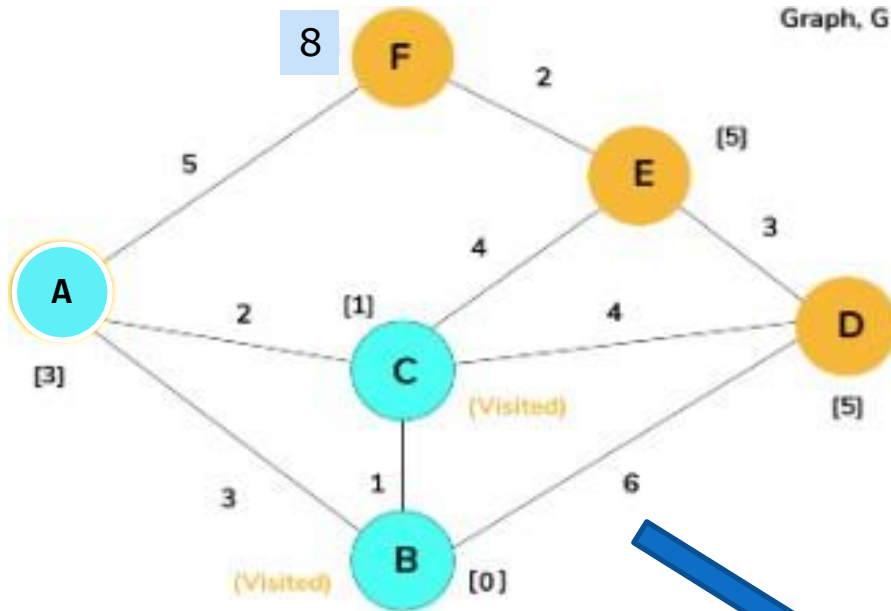
Step-5: Select next vertex with smallest cost from the unvisited list. (A)



- For neighbor F: cost = Minimum(∞ , 3+5) = 8

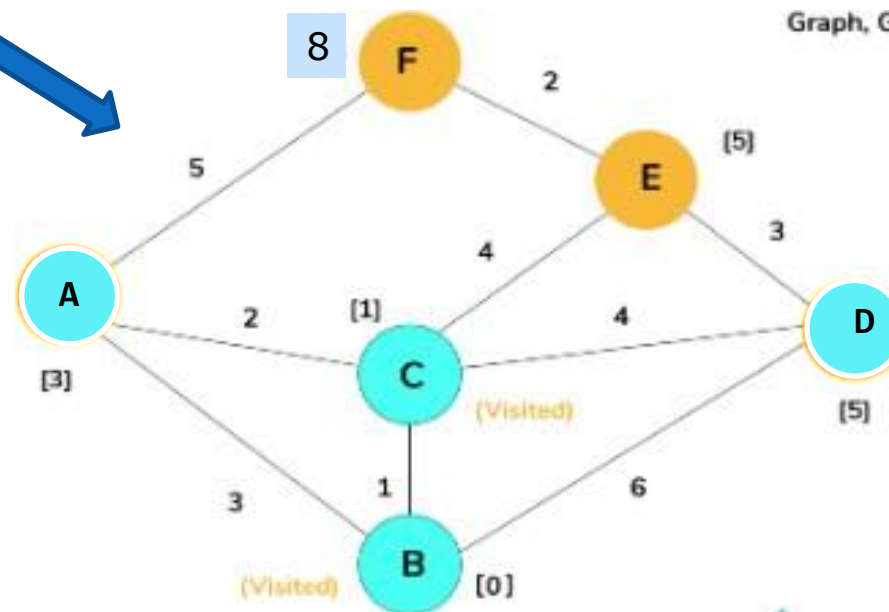
Step-6

Dijkstra's Algorithm



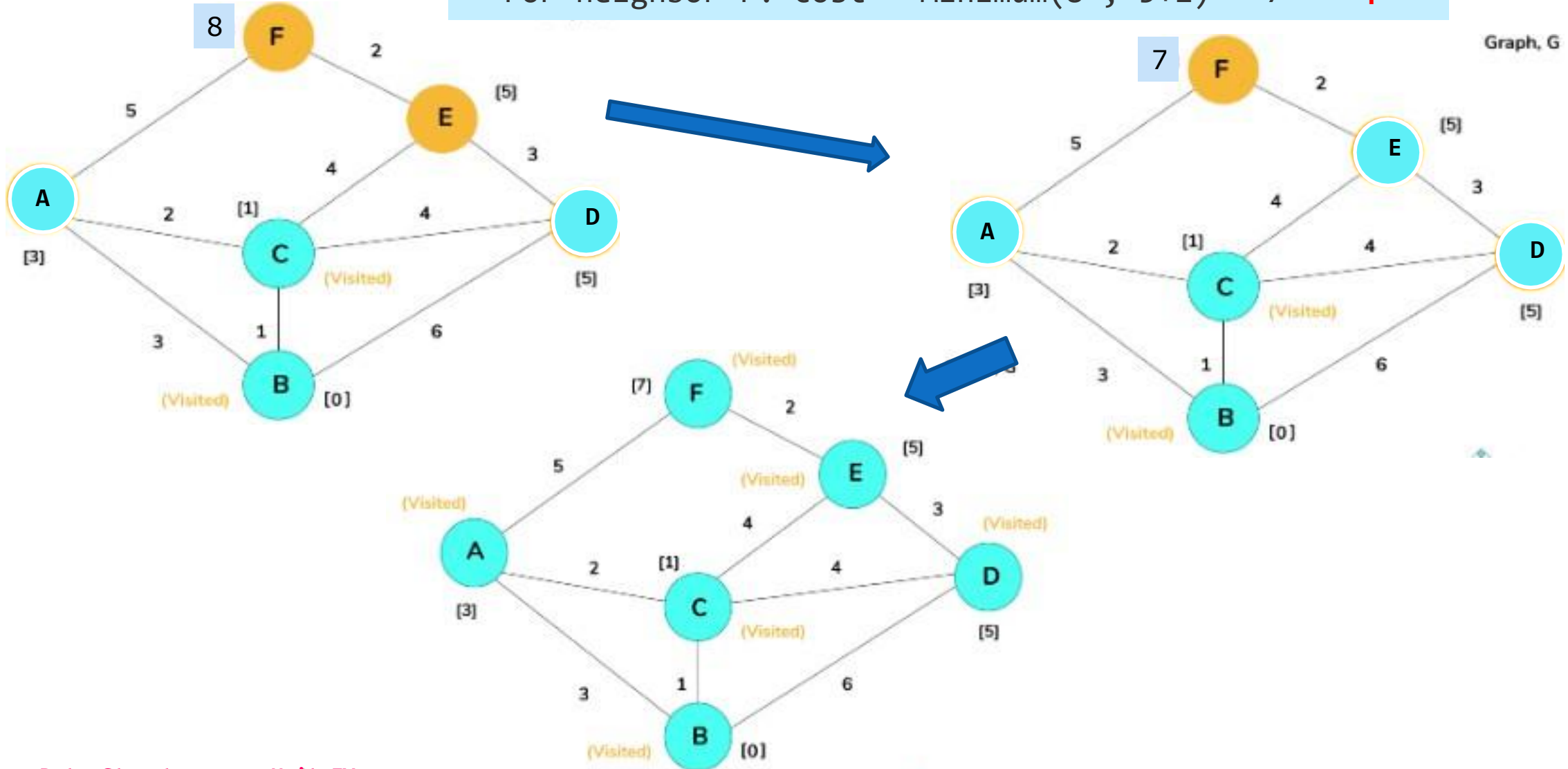
Step-7: Select next vertex with smallest cost from the unvisited list. **(D)**

- For neighbor E: cost = Minimum(5 , 3+5) = 8 **Step-8**



Step-9: Select next vertex with smallest cost from the unvisited list. (E)

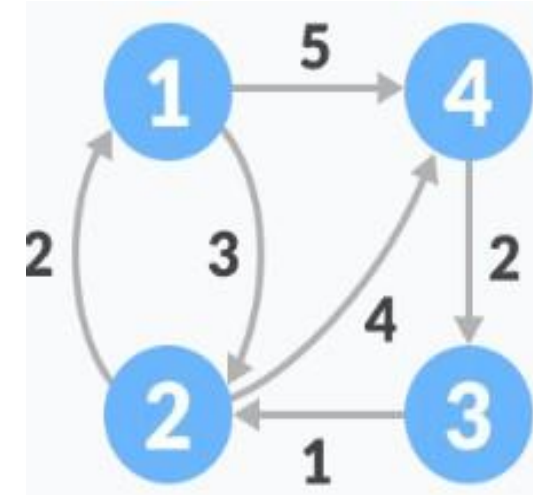
- For neighbor F: cost = Minimum(8 , 5+2) = 7 **Step-10**



Floyd-Warshall's algorithm (All pair shortest path Algorithm)

Step-1: Create a matrix A^0 of dimension $n \times n$ where n is the number of vertices.

Step-2: Each cell $A[i][j]$ is filled with the distance from the i^{th} vertex to the j^{th} vertex.
If there is no path from i^{th} vertex to j^{th} vertex, the cell is left as infinity.



Step-3: Now, create a matrix A^1 using matrix A^0 .
The elements in the first column and the first row are left as they are.
The remaining cells are filled in the following way.

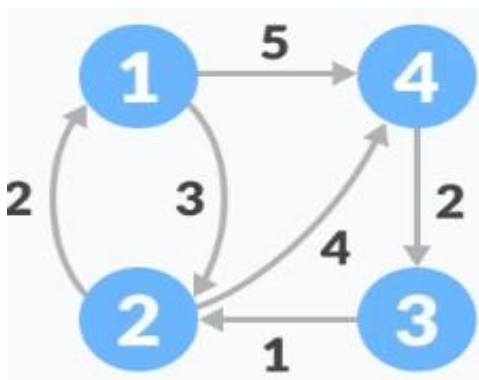
Let k be the intermediate vertex in the shortest path from source to destination.
(k starts from the first vertex).

$A[i][j]$ is filled with $(A[i][k] + A[k][j])$ if $(A[i][j] > A[i][k] + A[k][j])$

If the direct distance from the source to the destination is greater than the path through the vertex k , then the cell is filled with $A[i][k] + A[k][j]$.

Floyd-Warshall's algorithm

Step-1 and Step-2:



$$A^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

$$A^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & 4 \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & 9 & 4 \\ \infty & 1 & 0 & 8 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Step-3

Create a matrix A^1 using matrix A^0 . The elements in the first column and the first row are left as they are.

$$\begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & 4 \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{matrix}$$

Calculate the distance from the source vertex to destination vertex through the vertex k (In this step, k is vertex 1).

For $A^1[2, 3]$, the direct distance from vertex 2 to 3 is ∞ (no edge) and the sum of the distance from vertex 2 to 3 through vertex 1 (from vertex 2 to 1 and from vertex 1 to 4, 4 to 3) is 9.

Since $9 < \infty$, $A^0[2, 3]$ is filled with 9.

Floyd-Warshall's algorithm

Step-3 contd

A^2 is created using A^1 . The elements in the second column and the second row are left as they are.

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & & \\ 2 & 0 & 9 & 4 \\ & 1 & 0 & \\ & \infty & & 0 \end{bmatrix} \end{matrix}$$

Calculate the distance from the source vertex to destination vertex through the vertex k (In this step, k is vertex 2).

$$A^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & & \\ 2 & 0 & 9 & 4 \\ & 1 & 0 & \\ & \infty & & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 9 & 5 \\ 2 & 0 & 9 & 4 \\ 3 & 3 & 1 & 0 & 5 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Step-3 contd

A^3 is created using A^2

$$A^3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & \infty & \\ & 0 & 9 & \\ \infty & 1 & 0 & 8 \\ & & 2 & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 9 & 5 \\ 2 & 0 & 9 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

Step-3 contd

A^4 is created using A^3

$$A^4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & & & 5 \\ & 0 & & 4 \\ & & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 7 & 5 \\ 2 & 0 & 6 & 4 \\ 3 & 1 & 0 & 5 \\ 5 & 3 & 2 & 0 \end{bmatrix} \end{matrix}$$

A^4 gives the shortest path between each pair of vertices.